Step 5: turn your project into a working application

Product V1 - beginner reference guide

SC11 Water Quality Potability is a worked teaching example. The screenshots illustrate the instructor's journey; students use them to understand the process and then build their own allotted project.

The appendix contains SC11-specific code and a report outline for learning. Read how they work, choose only ideas suitable for your project, and create your own inputs, method, tests, outputs and documentation. A project using fuzzy rules, optimisation or another technique need not use SC11's ANN or filenames.

Continue from the completed Step 4 branch. Reuse the saved model or working controller; connect it to an interface that a user can operate.

Submission deadline: **8 November 2026, 11:59:59 PM IST.**

Screenshots in this guide are visual references, not instructions to reproduce identical screenshots. Record evidence that genuinely demonstrates your own project.

What should exist at the end?

- A working input interface connected to your actual saved method.
- Checks for normal, boundary, missing and invalid input, with clear error messages.
- Readable outputs with labels, units and the correct project limitations.
- Reproducible launch instructions and the dependencies needed on another computer.
- A completed project report, recorded test observations and suitable evidence of the actual work.
- A meaningful commit and published branch, followed by your team's Step 5 portal submission.

How the remaining steps fit together

- Step 5 - 8 November: Product V1, a complete input-to-output application.
- Step 6 - 15 November: systematic testing, validation, error analysis and fixes.
- Step 7 - 22 November: final documentation, reproducibility and demonstration preparation.
- Step 8 - 29 November: final demonstration, explanation of contributions and completion evidence.

All dates are in 2026 and close at 11:59:59 PM IST. Detailed Step 6-8 instructions will be issued before activation.

Adapt the interface to your project

- SC01: room conditions and tariff in; cooling, fan and energy action out.
- SC02: load, fabric and dirt inputs; wash settings and resource estimates out.
- SC03: documented student indicators in; risk indicator and supportive action out.
- SC04: traffic scenario in; signal plan and measured simulation outputs out.
- SC05: courses, rooms and constraints in; timetable, unplaced requests and clashes out.
- SC06: locations and constraints in; route, distance and constraint results out.
- SC07: soil/weather inputs in; irrigation recommendation and explanation out.
- SC08: machine measurements in; failure warning and priority explanation out.
- SC09: permitted applicant features in; decision-support output and error limits out.
- SC10: appliance requirements and tariff in; schedule, cost and constraint results out.

The file names, algorithms and tests may differ by project. Use the reference to understand the workflow and write your team's own implementation.

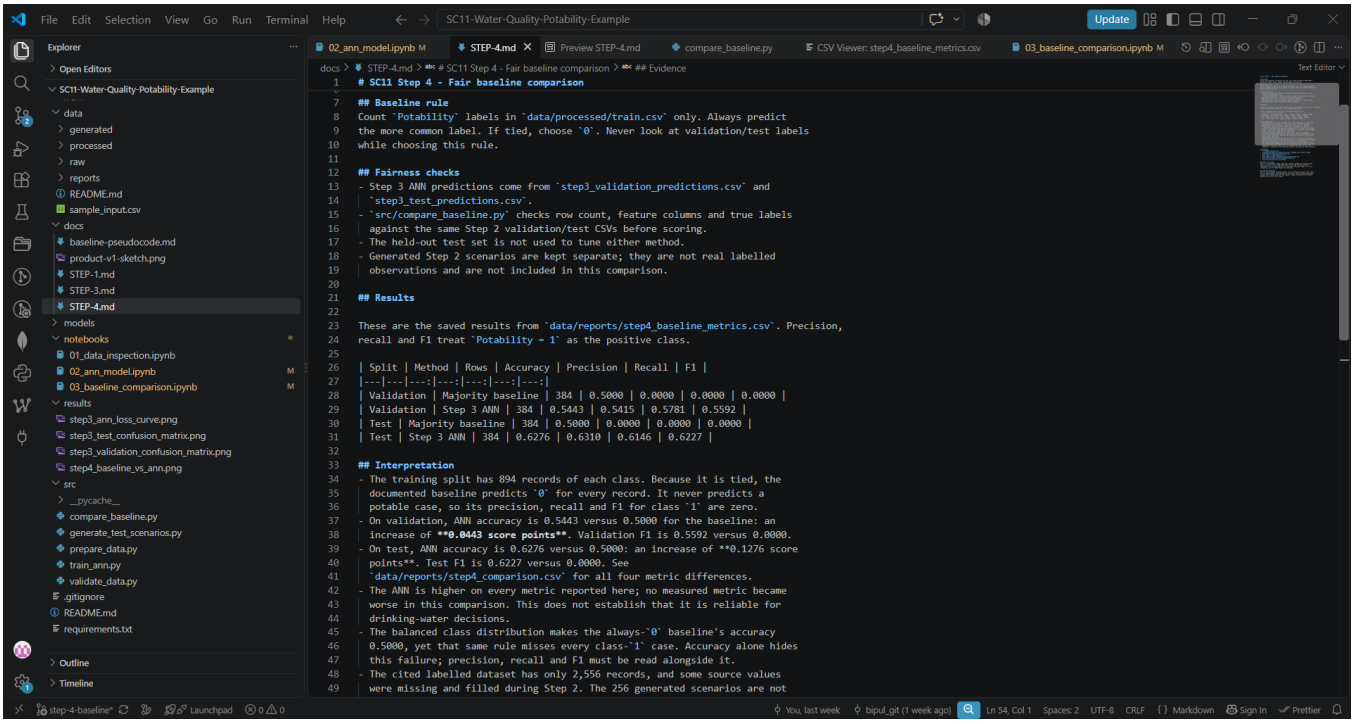
Action 1: Open the completed Step 4 project

Why and how - Repository and clone: A repository is the project and its saved history. Cloning makes a working copy on your computer. Open the completed Step 4 work so the new app uses your existing method.

Expected evidence: Explorer shows the project folders and completed Step 4 branch.

1. Open VS Code. Choose **File > Open Folder** and select your existing project repository folder. For the instructor example this is SC11-Water-Quality-Potability-Example.

Action 1 - illustration 1: Open the existing SC11 project and inspect the completed Step 4 branch.



2. If the project is not on this computer, open the Command Palette with Ctrl+Shift+P, choose **Git: Clone**, enter your team's repository URL, choose a folder and click Open.
3. Use the branch selector at the bottom left to open the branch containing the completed Step 4 work. Use Sync Changes to bring your local copy up to date.
4. Check that Step 3's saved model and metadata, data splits and Step 4 comparison files are present. Review the dashboard feedback and list any corrections still needed.

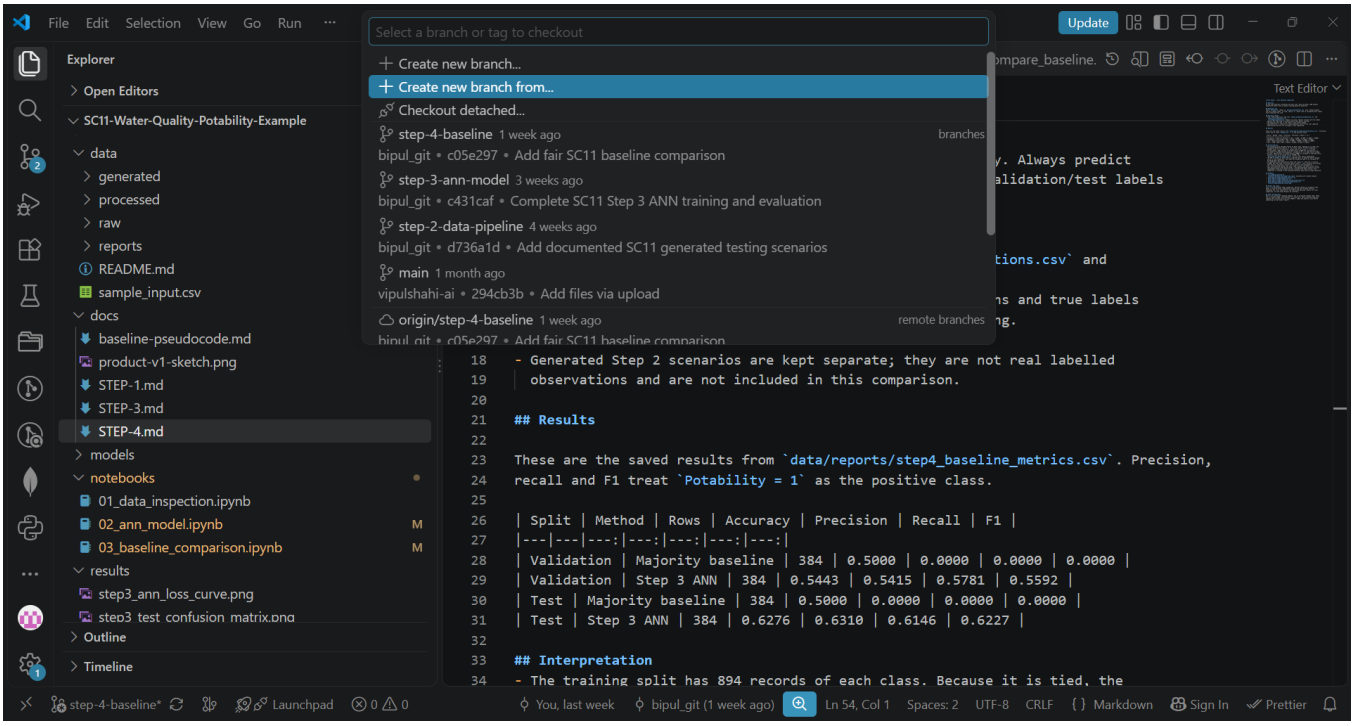
Action 2: Create the Product V1 branch from Step 4

Why and how - Branch: A branch is a separate line of project work. Starting it from your completed Step 4 carries those files forward while you develop Step 5 without mixing unfinished work into the earlier branch.

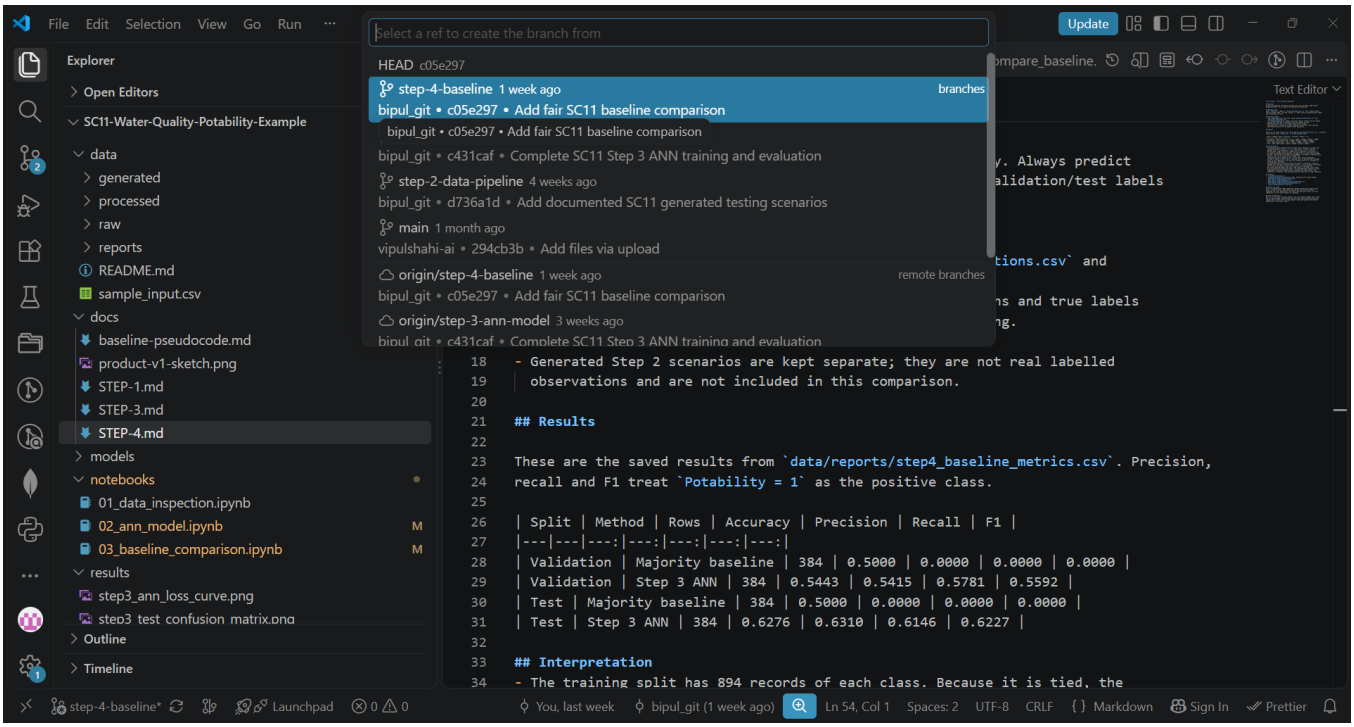
Expected evidence: The new branch name and inherited Step 4 files are visible.

1. Save your files. Review any uncommitted changes before switching branches; keep unrelated work separate.
2. Click the current branch name at the bottom left and choose **Create new branch from...**. Select the branch or remote branch containing your completed Step 4 work.

Action 2 - illustration 1: Click the branch name and choose Create new branch from...

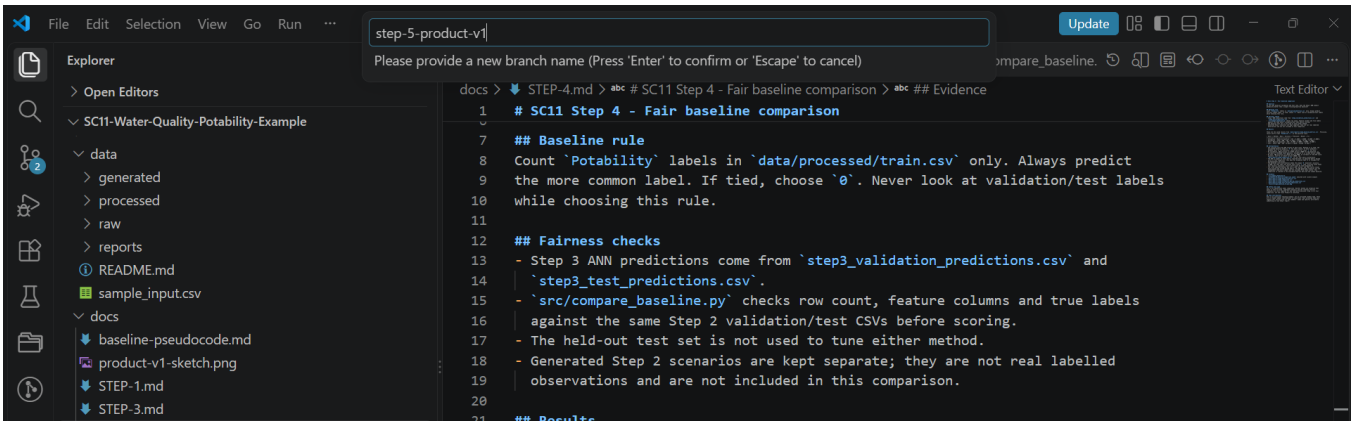


Action 2 - illustration 2: Select the Step 4 branch as the source of the new branch.



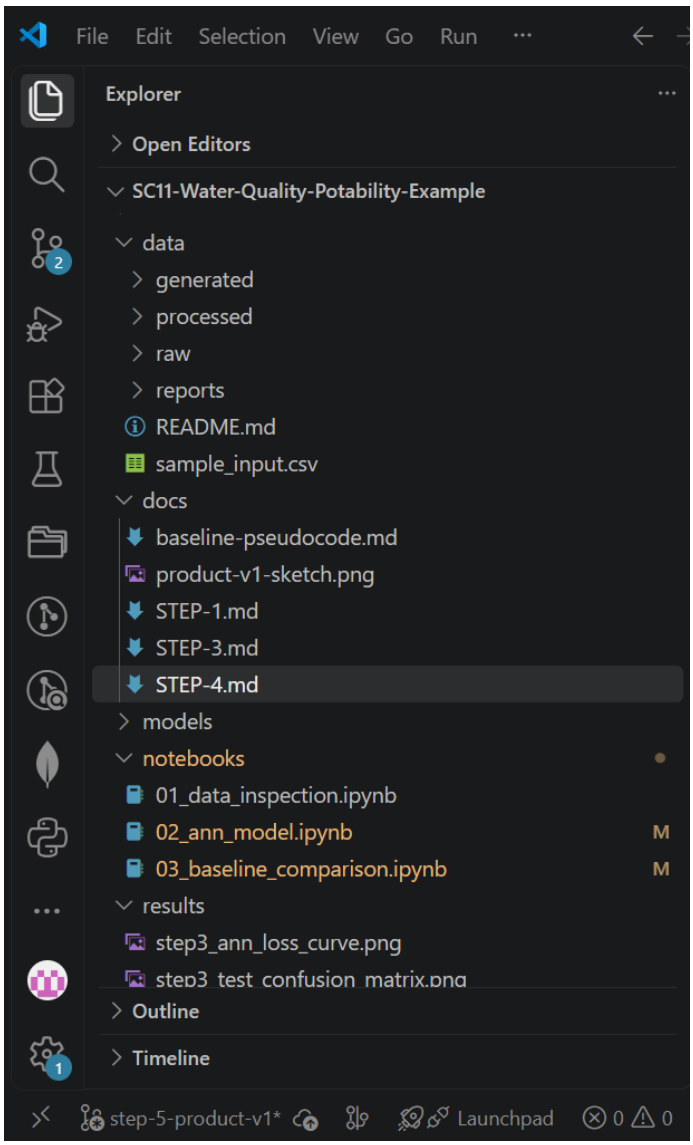
3. For SC11, select **step-4-baseline**. Another team's branch may have a different name, such as step-4-fuzzy-controller. Then enter **step-5-product-v1** and press Enter.

Action 2 - illustration 3: Enter the new branch name step-5-product-v1 and press Enter.



4. Confirm step-5-product-v1 is now shown at the bottom left. The earlier model, reports and source files should still be visible.

Action 2 - illustration 4: Confirm step-5-product-v1 is shown in the lower-left status bar.



Action 3: Define the user task and file plan

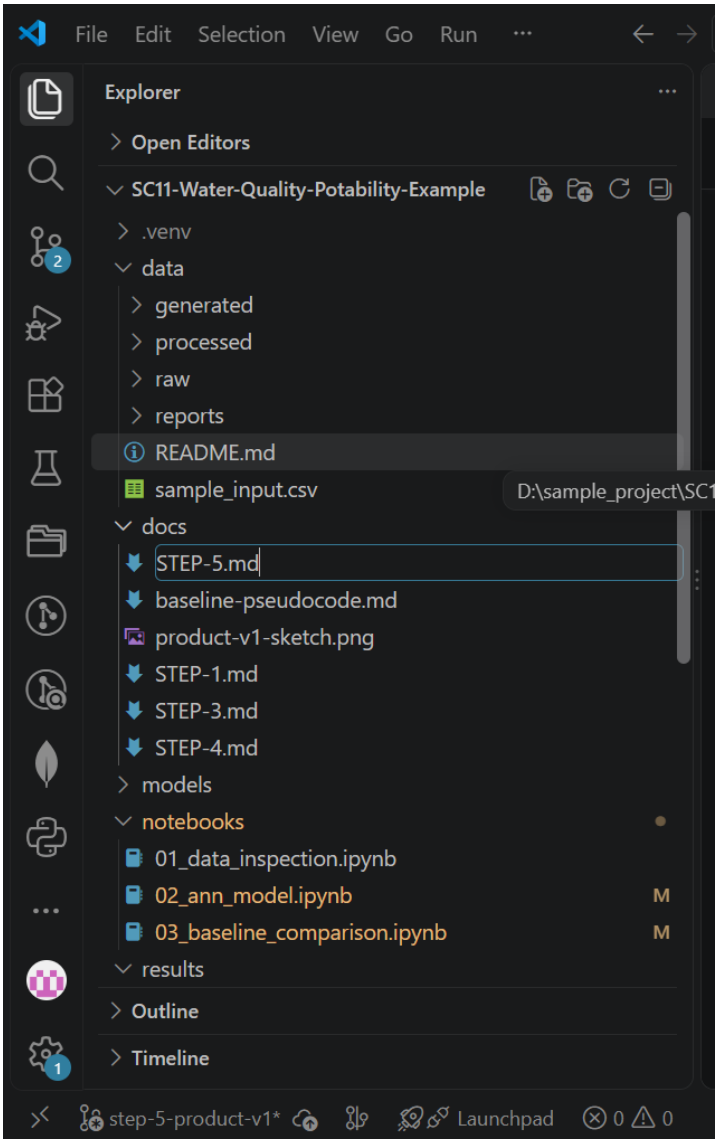
Why and how - Product V1: A first usable version lets someone provide inputs and receive an understandable result. Define the user and task before coding, so each field and output has a clear purpose.

Expected evidence: The report states the user, input, method, output and file plan.

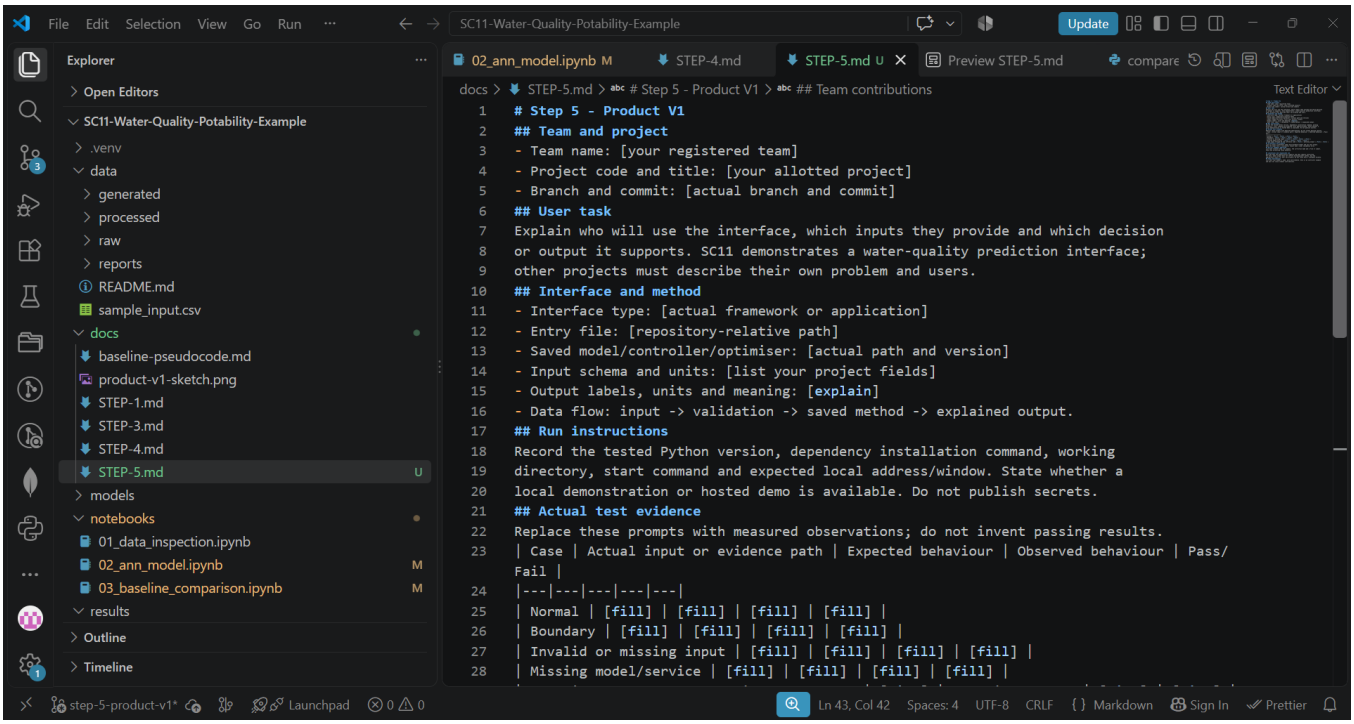
1. Decide which person uses your app and what they enter. Explain what the output means in that person's context.
2. For SC11, a user enters nine water measurements and sees the saved ANN's predicted class. This is an educational demonstration, not a certificate that water is safe to drink.

3. Create a Step 5 report for your team's design and evidence. For SC11 the example report is docs/STEP-5.md; use the example headings as inspiration and write your own details.

Action 3 - illustration 1: Create STEP-5.md inside the existing docs folder.



Action 3 - illustration 2: The initial report template is open; replace its prompts with actual project details.



4. Plan the flow: input form - validation - saved model/controller/optimiser - readable output. Keep training outside the prediction button.

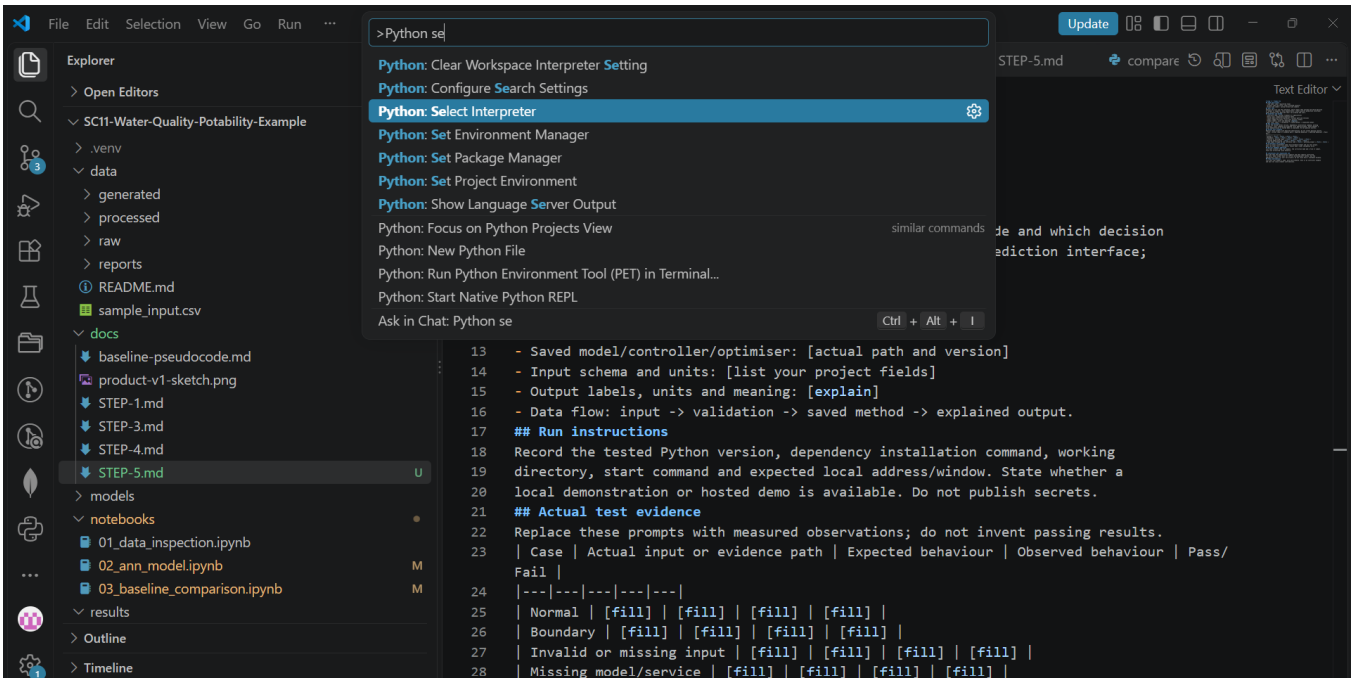
Action 4: Prepare the Python environment

Why and how - Environment and dependencies: An environment holds the Python interpreter and packages used by a project. Dependencies are those packages. Selecting the environment and recording versions helps the app run consistently and load its saved model.

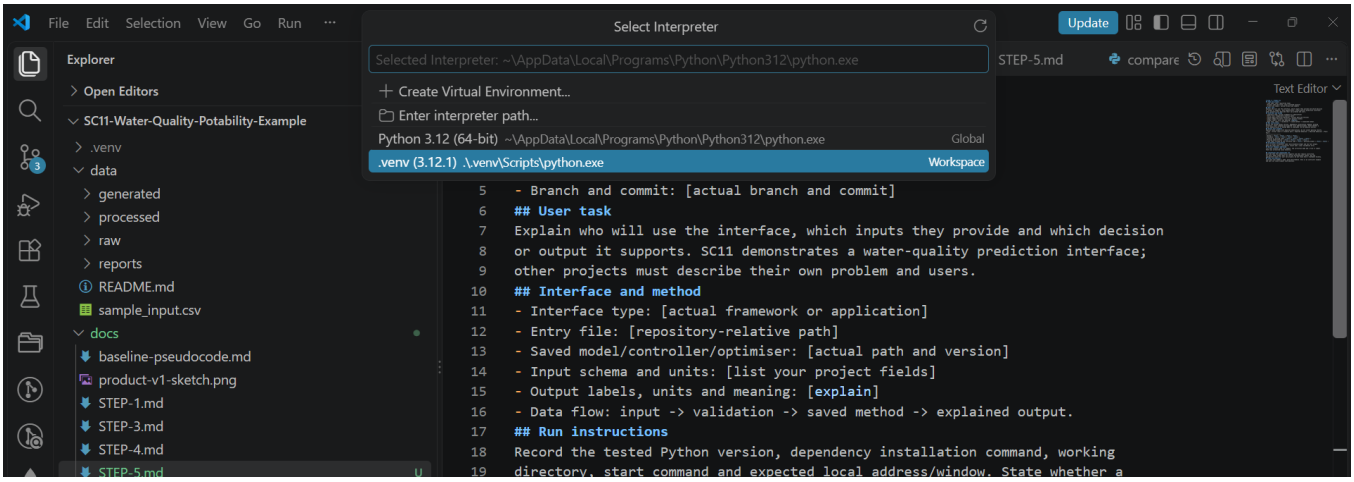
Expected evidence: Terminal shows the interpreter and successful dependency installation.

1. Press Ctrl+Shift+P, choose **Python: Select Interpreter** and select the same .venv interpreter that successfully ran Step 3. Using a different scikit-learn version can prevent a saved model from loading.

Action 4 - illustration 1: Use the Command Palette to find Python: Select Interpreter.

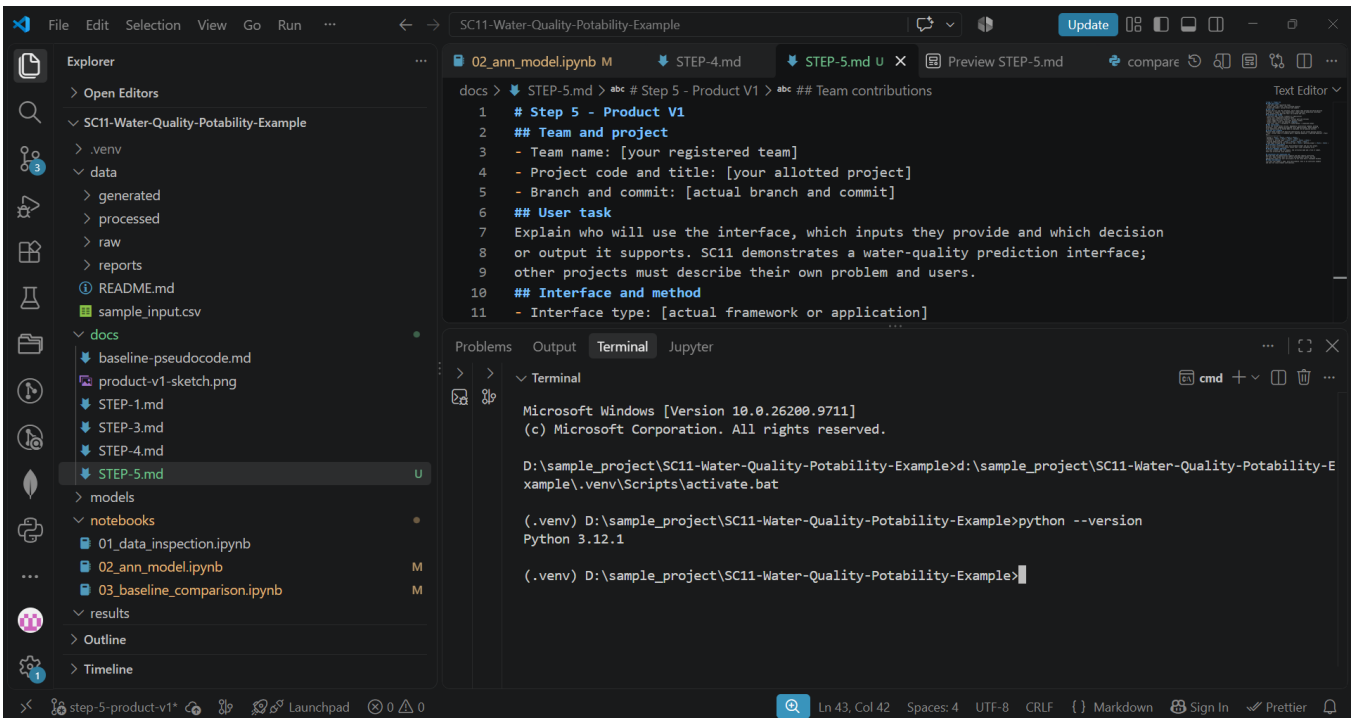


Action 4 - illustration 2: Choose the project .venv interpreter used for the earlier model.



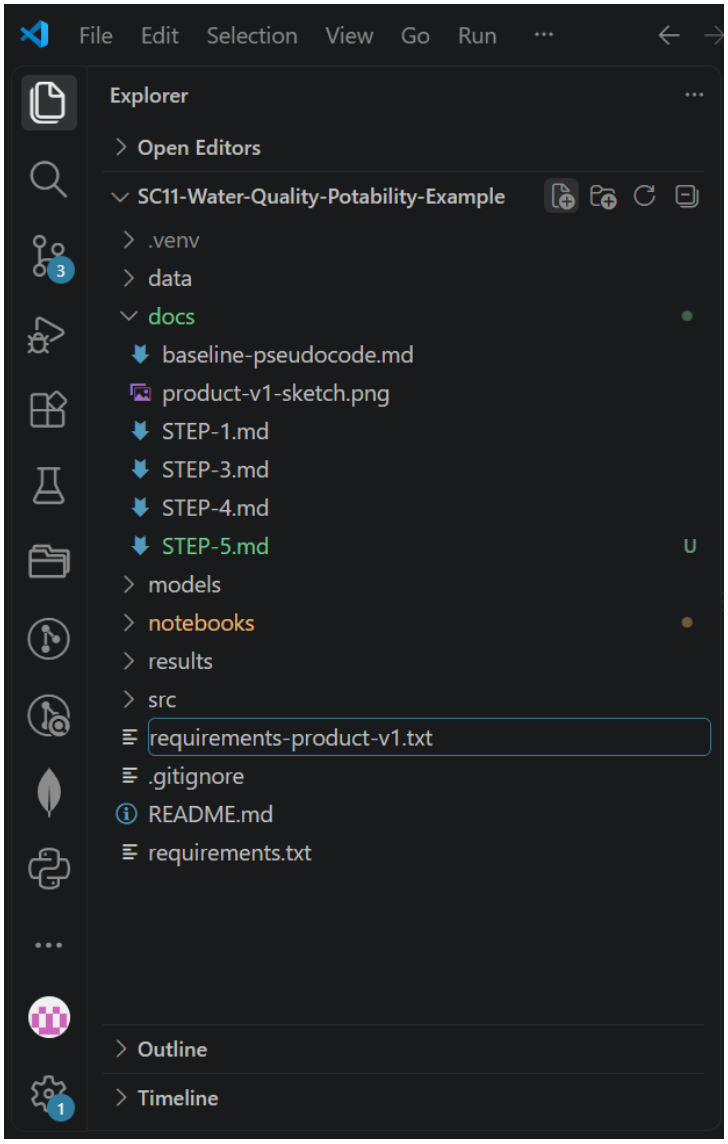
2. Choose **Terminal > New Terminal**. Check the prompt is inside the project root. Run **python --version** and confirm the selected environment.

Action 4 - illustration 3: Open the terminal and check python --version in the project environment.

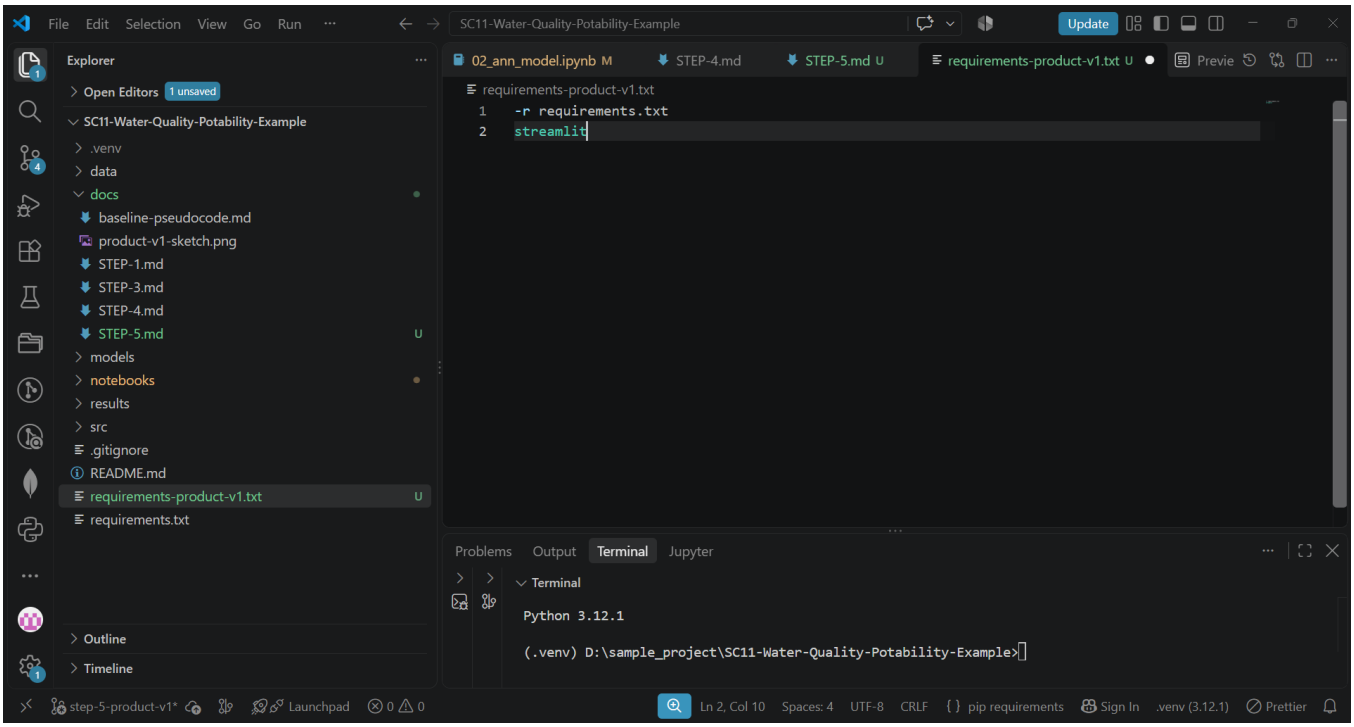


3. Record the packages your own application needs. Appendix A shows how the SC11 example extends its existing requirements with Streamlit; your project may need different packages or a different setup.

Action 4 - illustration 4: Create requirements-product-v1.txt in the project root.

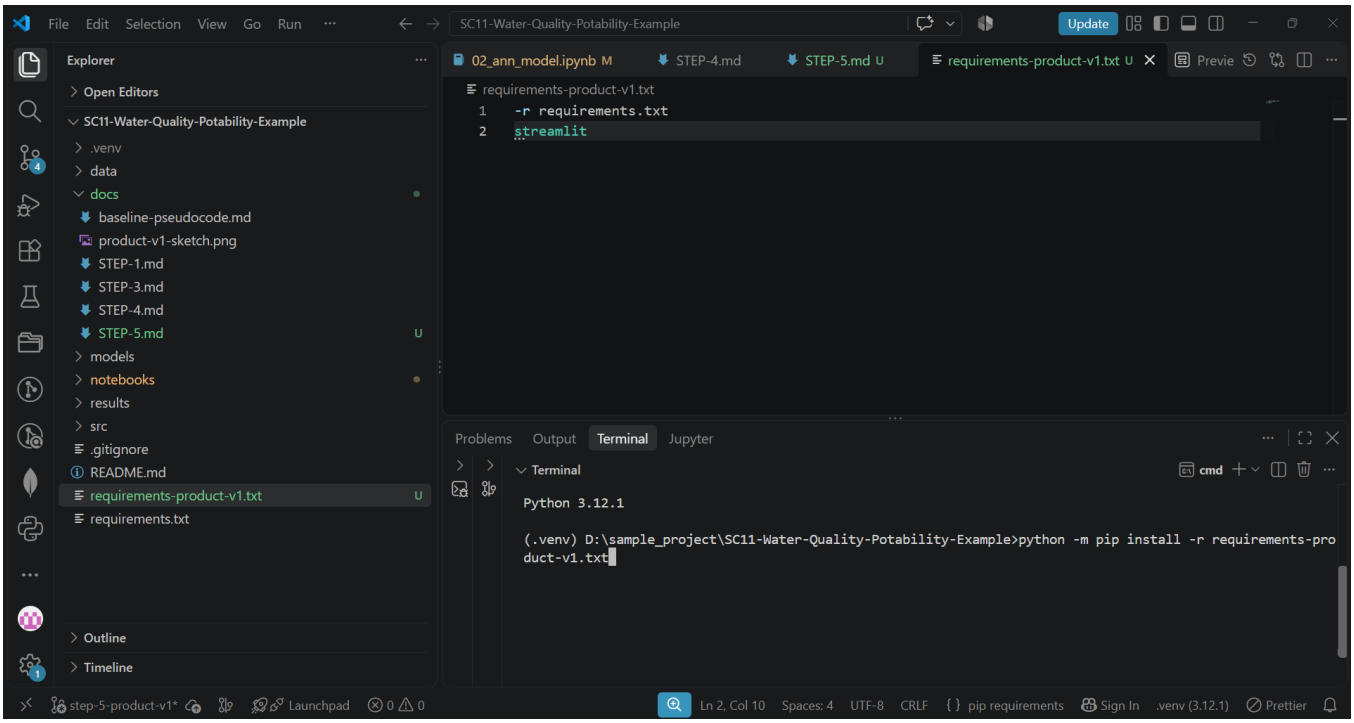


Action 4 - illustration 5: The dependency file includes the existing requirements and Streamlit.

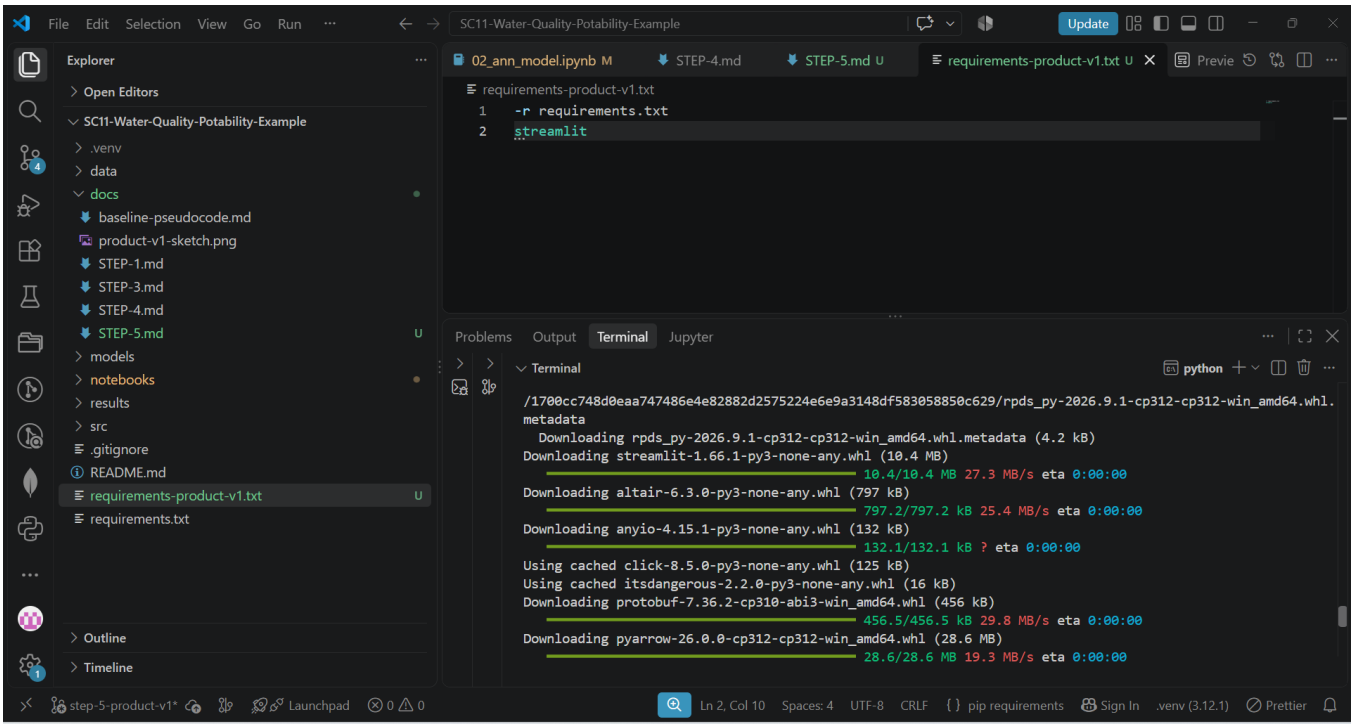


4. For SC11, create requirements-product-v1.txt and run **python -m pip install -r requirements-product-v1.txt**. For another project, use its own dependency file and installation command. Resolve errors before continuing.

Action 4 - illustration 6: Run the dependency installation command from the project root.



Action 4 - illustration 7: Packages are downloading. Wait for a successful completion message before continuing.



5. For another framework, install and record that project's actual dependencies. Streamlit is the SC11 reference interface, not a mandatory framework for all teams.

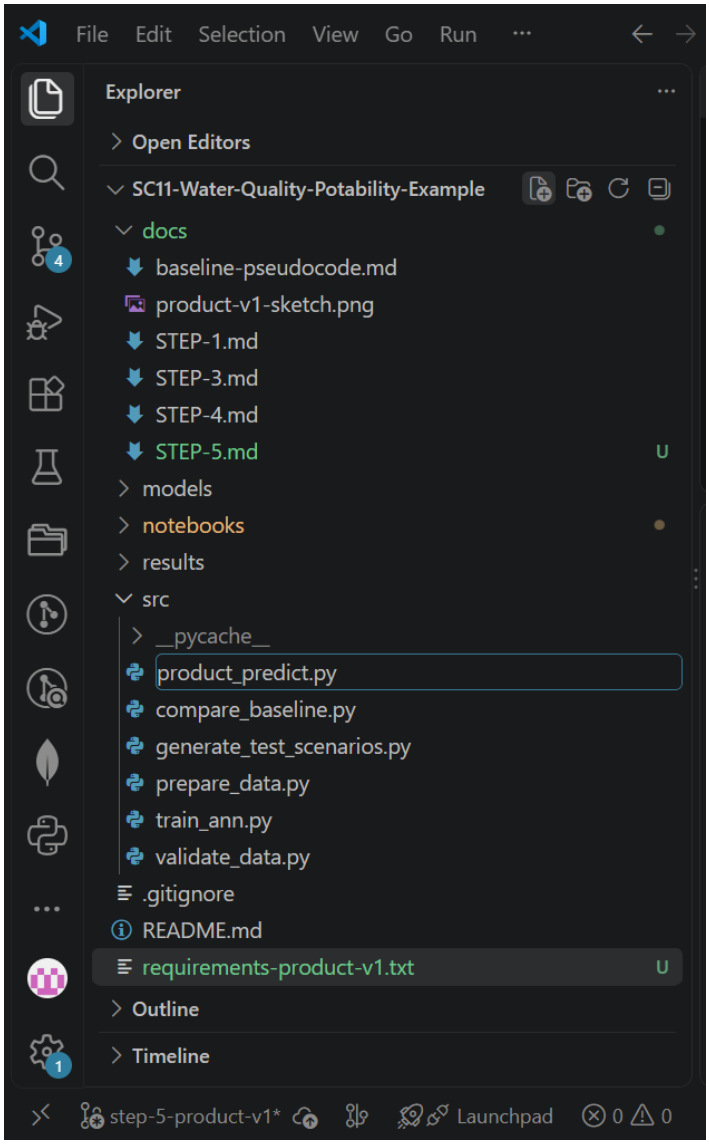
Action 5: Add the inference module

Why and how - Inference and input validation: Training builds a model; inference uses the saved model to predict for a new input. Validation checks that required inputs have acceptable types and ranges before prediction. Keep the feature names and order consistent with training.

Expected evidence: The source shows the schema, validation and saved-model inference.

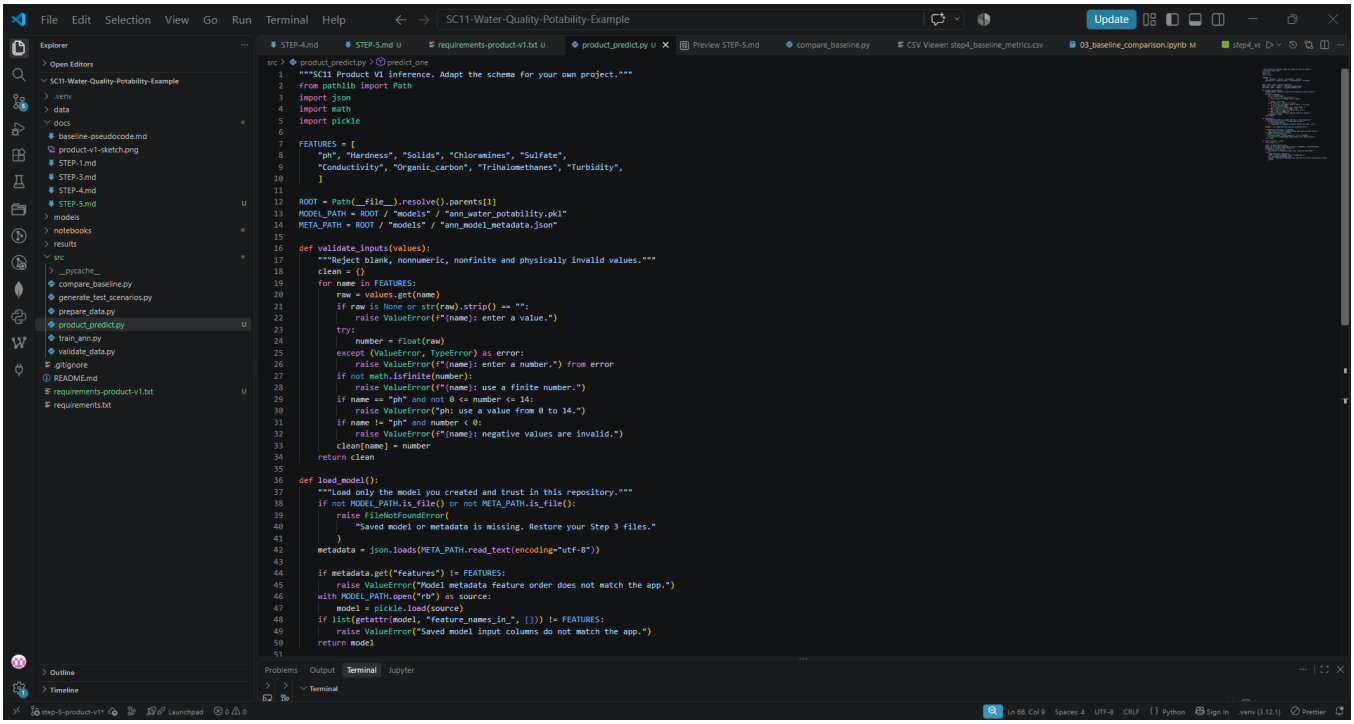
1. Plan the module that connects your interface to your own method. In SC11, the example file is created as **src/product_predict.py**.

Action 5 - illustration 1: Create product_predict.py inside src.



2. Study Appendix B to see how SC11 validates water fields and loads an ANN. Write or adapt logic for your own input fields, rules, controller or model and outputs; SC11 field names and paths are not universal.

Action 5 - illustration 2: Review the feature names and input validation in the SC11 inference module.

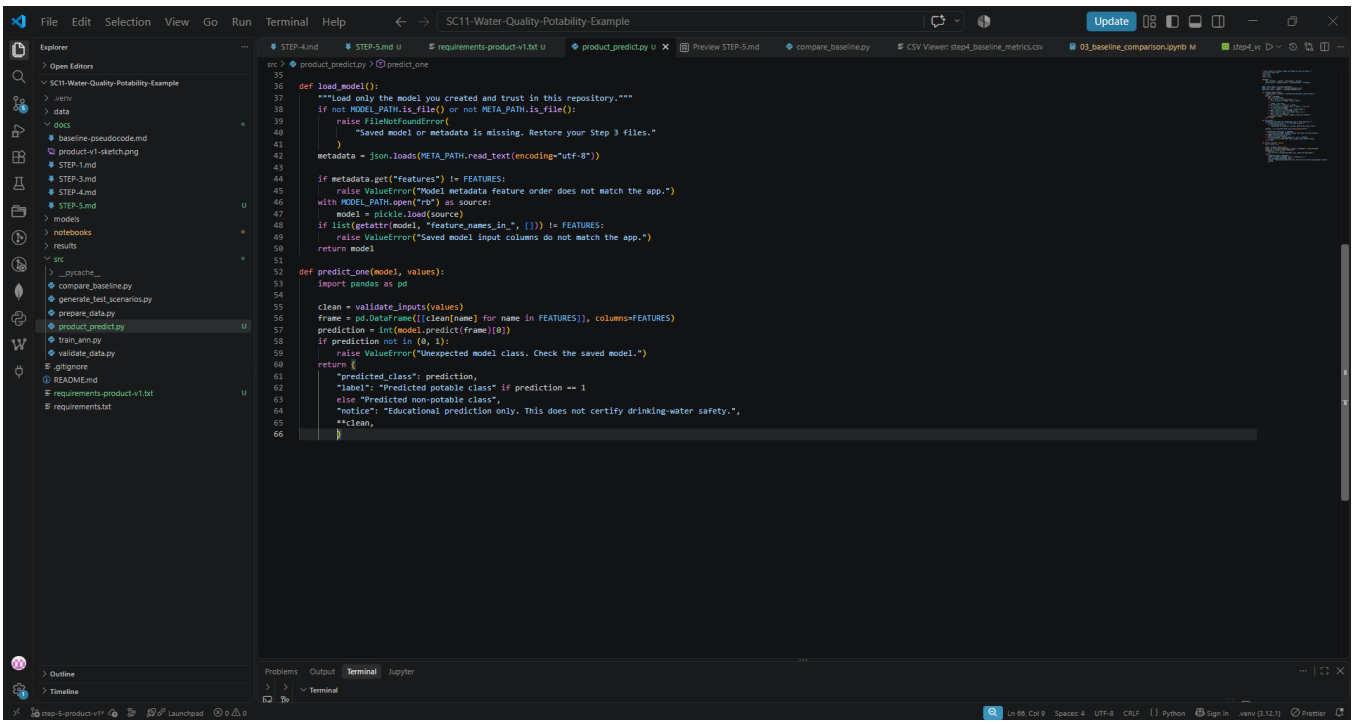


3. For SC11, keep the nine feature names and their order identical to ann_model_metadata.json. The saved model is a pipeline that already contains its fitted scaler.

4. In SC11, validation rejects blanks, text, NaN, infinity, pH outside 0-14 and negative measurements. Other projects must decide their own valid ranges and constraints. Passing validation does not prove a value is typical of training data.

5. Save with Ctrl+S. The module calls predict() and does not fit or retrain the model. Load only the saved model you created and trust.

Action 5 - illustration 3: Review the saved-model loading and predict_one function.



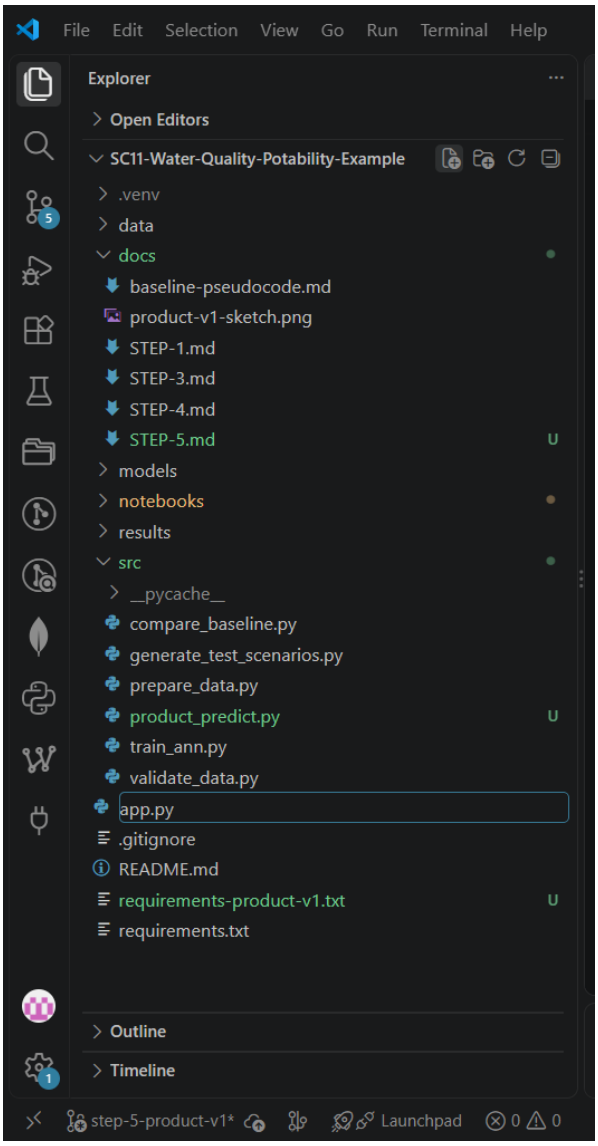
Action 6: Add the interface

Why and how - Interface: The interface is the part a user sees and operates. It collects inputs, calls your project method, and presents a result or a helpful error. It should not pretend a fixed example is a real prediction.

Expected evidence: app.py is at the root and contains the form and prediction call.

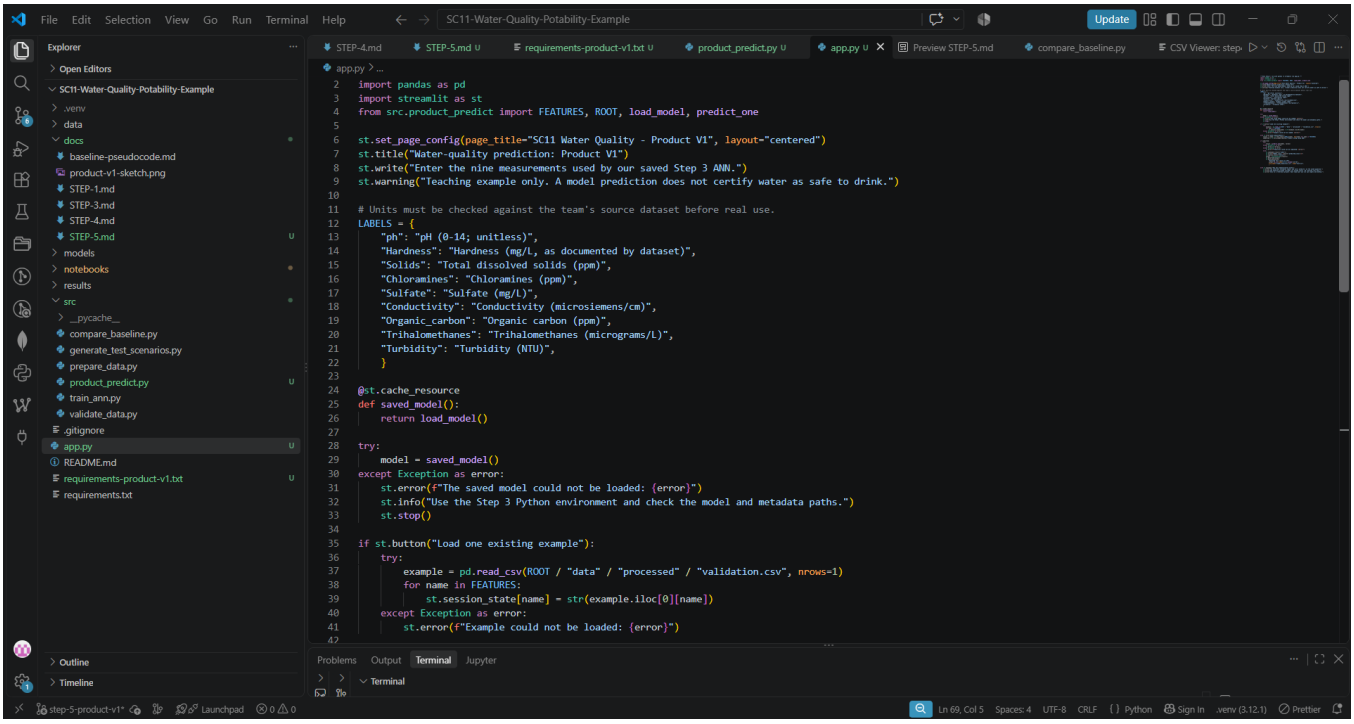
1. Create an entry file appropriate to your chosen interface. SC11 uses **app.py** at the repository root; your project may use a different filename or framework.

Action 6 - illustration 1: Create app.py in the repository root, not inside src.



2. Study Appendix C to see how the SC11 form connects inputs to its method and displays a result. Design your own fields, decisions and messages around your project requirements.

Action 6 - illustration 2: Review the interface setup, input labels and saved-model loading.



```python
import pandas as pd
import streamlit as st
from src.product_predict import FEATURES, ROOT, load_model, predict_one

st.set_page_config(page_title="SC11 Water Quality - Product V1", layout="centered")
st.title("Water-quality prediction: Product V1")
st.write("Enter the nine measurements used by our saved Step 3 ANN.")
st.warning("Teaching example only. A model prediction does not certify water as safe to drink.")

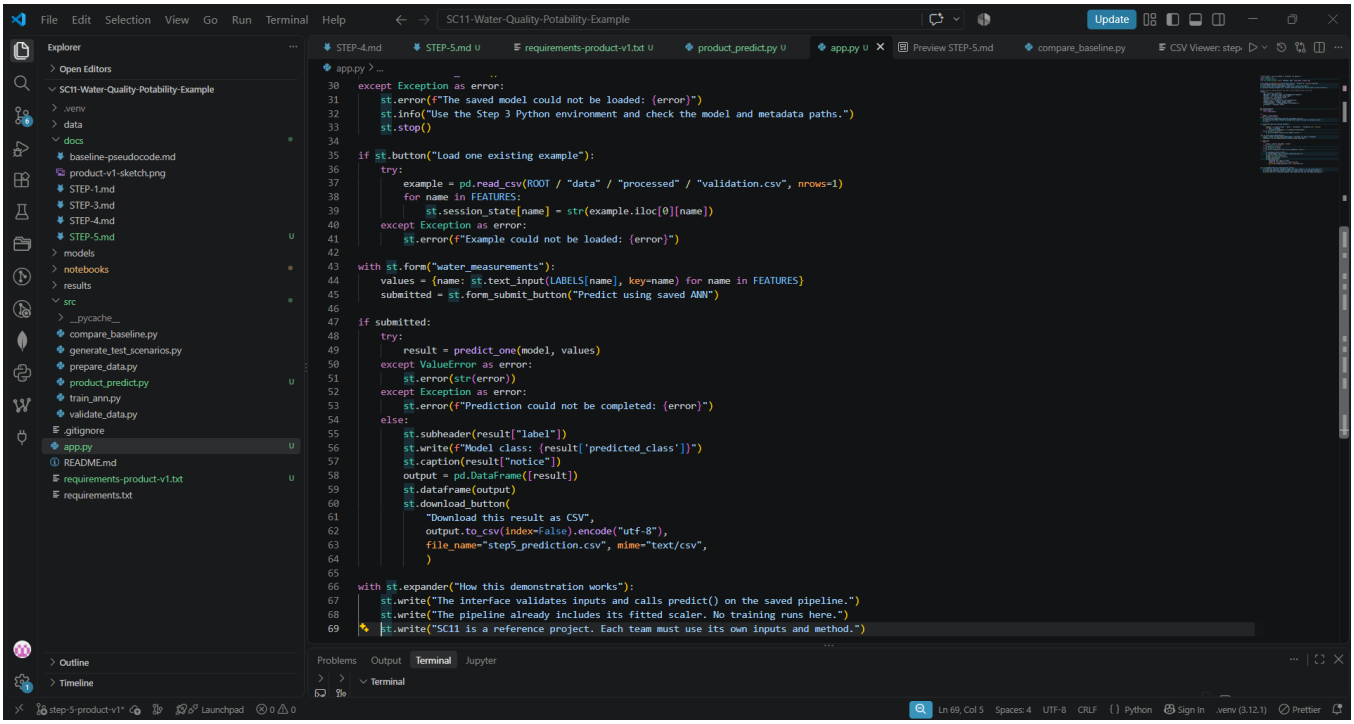
# Units must be checked against the team's source dataset before real use.
LABELS = {
    "ph": "pH (0-14; unitless)",
    "Hardness": "Hardness (mg/L, as documented by dataset)",
    "Solids": "Total dissolved solids (ppm)",
    "Chloramines": "Chloramines (ppm)",
    "Sulfate": "Sulfate (mg/L)",
    "Conductivity": "Conductivity (microsiemens/cm)",
    "Organic_carbon": "Organic carbon (ppm)",
    "Trihalomethanes": "Trihalomethanes (micrograms/L)",
    "Turbidity": "Turbidity (NTU)",
    }

@st.cache_resource
def saved_model():
    return load_model()

try:
    model = saved_model()
except Exception as error:
    st.error(f"The saved model could not be loaded: {error}")
    st.info("Use the Step 3 Python environment and check the model and metadata paths.")
    st.stop()

if st.button("Load one existing example"):
    try:
        example = pd.read_csv(ROOT / "data" / "processed" / "validation.csv", nrows=1)
        for name in FEATURES:
            st.session_state[name] = str(example.iloc[0][name])
    except Exception as error:
        st.error(f"Example could not be loaded: {error}")
```

Action 6 - illustration 3: Review the form, prediction call and result display.



```python
except Exception as error:
    st.error(f"The saved model could not be loaded: {error}")
    st.info("Use the Step 3 Python environment and check the model and metadata paths.")
    st.stop()

if st.button("Load one existing example"):
    try:
        example = pd.read_csv(ROOT / "data" / "processed" / "validation.csv", nrows=1)
        for name in FEATURES:
            st.session_state[name] = str(example.iloc[0][name])
    except Exception as error:
        st.error(f"Example could not be loaded: {error}")

with st.form("water_measurements"):
    values = {name: st.text_input(LABELS[name], key=name) for name in FEATURES}
    submitted = st.form_submit_button("Predict using saved ANN")

if submitted:
    try:
        result = predict_one(model, values)
    except ValueError as error:
        st.error(str(error))
    except Exception as error:
        st.error(f"Prediction could not be completed: {error}")
    else:
        st.subheader(result["label"])
        st.write(f"Model class: {result['predicted_class']}")
        st.caption(result["notice"])
        output = pd.DataFrame([result])
        st.dataframe(output)
        st.download_button(
            "Download this result as CSV",
            output.to_csv(index=False).encode("utf-8"),
            file_name="step5_prediction.csv", mime="text/csv",
            )

with st.expander("How this demonstration works"):
    st.write("The interface validates inputs and calls predict() on the saved pipeline.")
    st.write("The pipeline already includes its fitted scaler. No training runs here.")
    st.write("SC11 is a reference project. Each team must use its own inputs and method.")
```

3. Keep field labels and units consistent with the source dataset. A credit-risk, scheduling or irrigation project must display its own fields and outputs.

4. Use a clear Predict, Recommend or Optimise button appropriate to your project. Show the actual method output rather than a fixed demonstration message.

5. Save the file. Read the labels and limitation notice as if you were a first-time user.

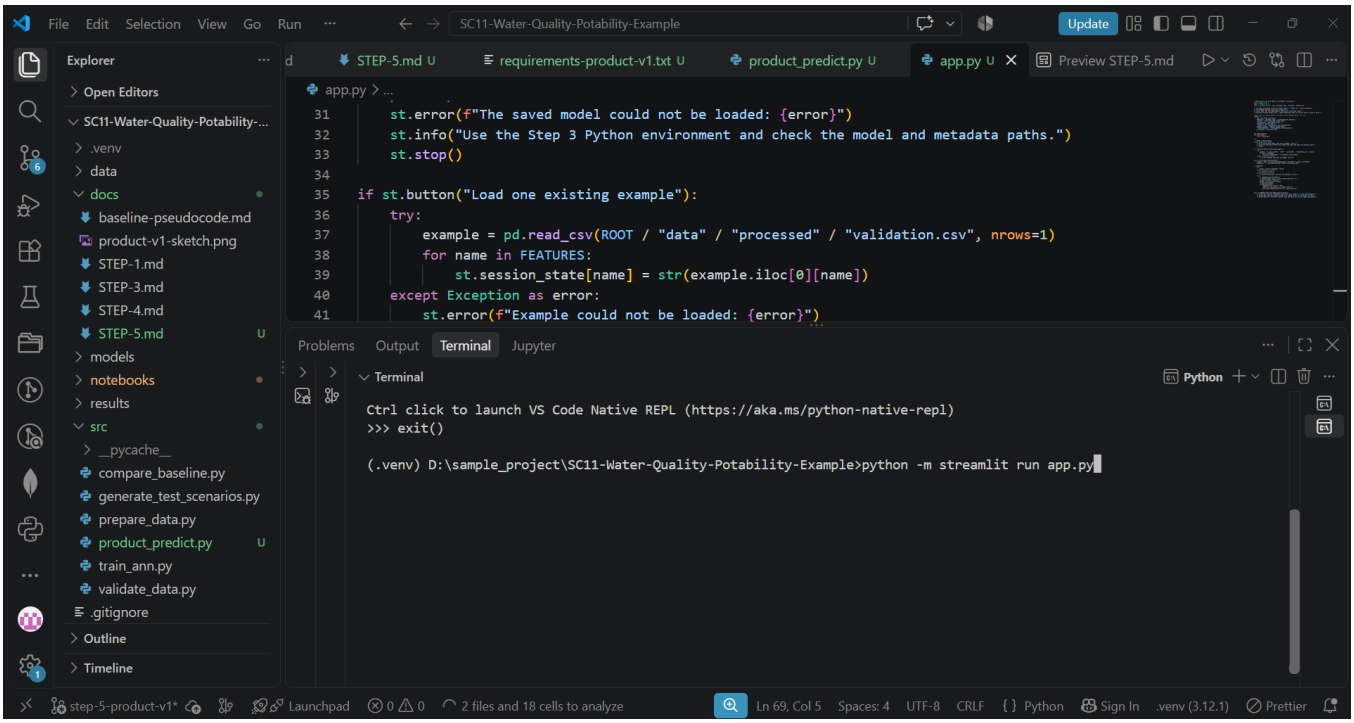
Action 7: Start Product V1

Why and how - Local app and terminal: A terminal runs commands in the project environment. Streamlit starts a local web app; localhost means it is running on your own computer. Keep the terminal running while you test the app.

Expected evidence: The terminal shows the Local URL and the app form opens.

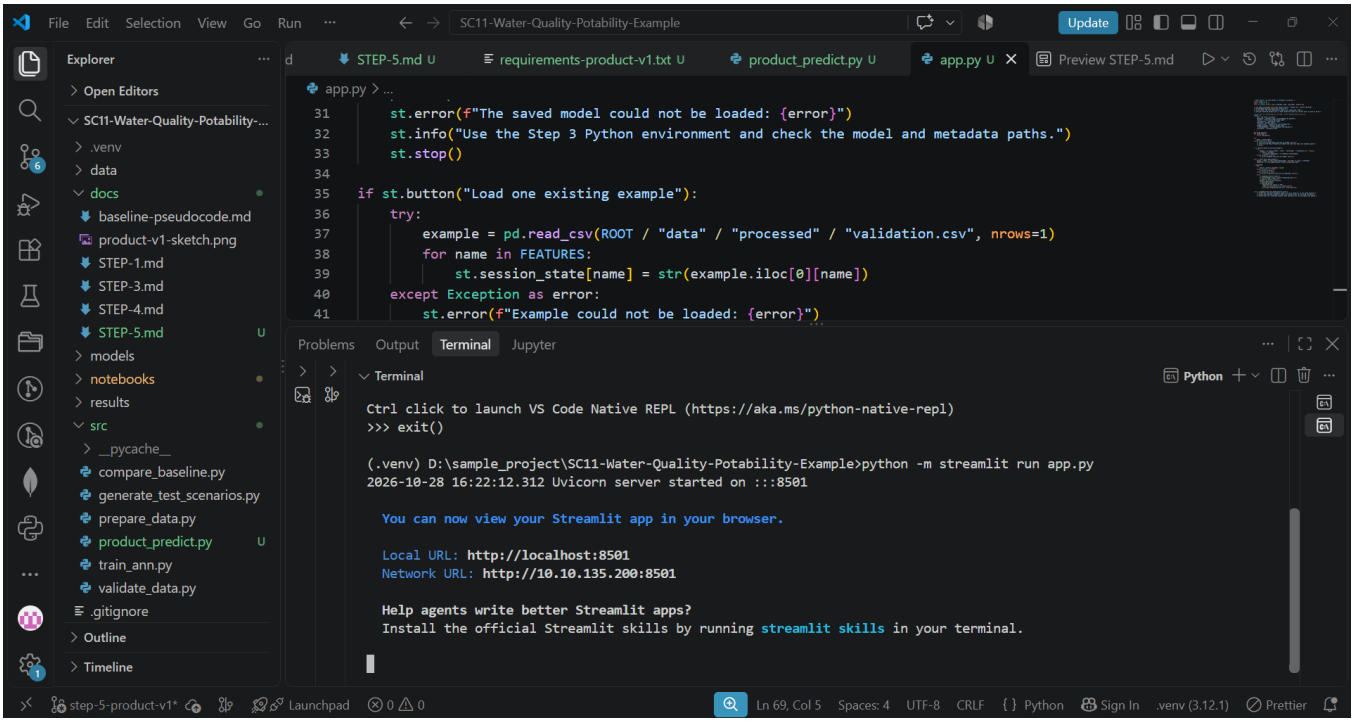
1. Open the terminal in your project root and start your chosen interface using its documented command. For the SC11 Streamlit example run **python -m streamlit run app.py**.

Action 7 - illustration 1: Run python -m streamlit run app.py in the project terminal.



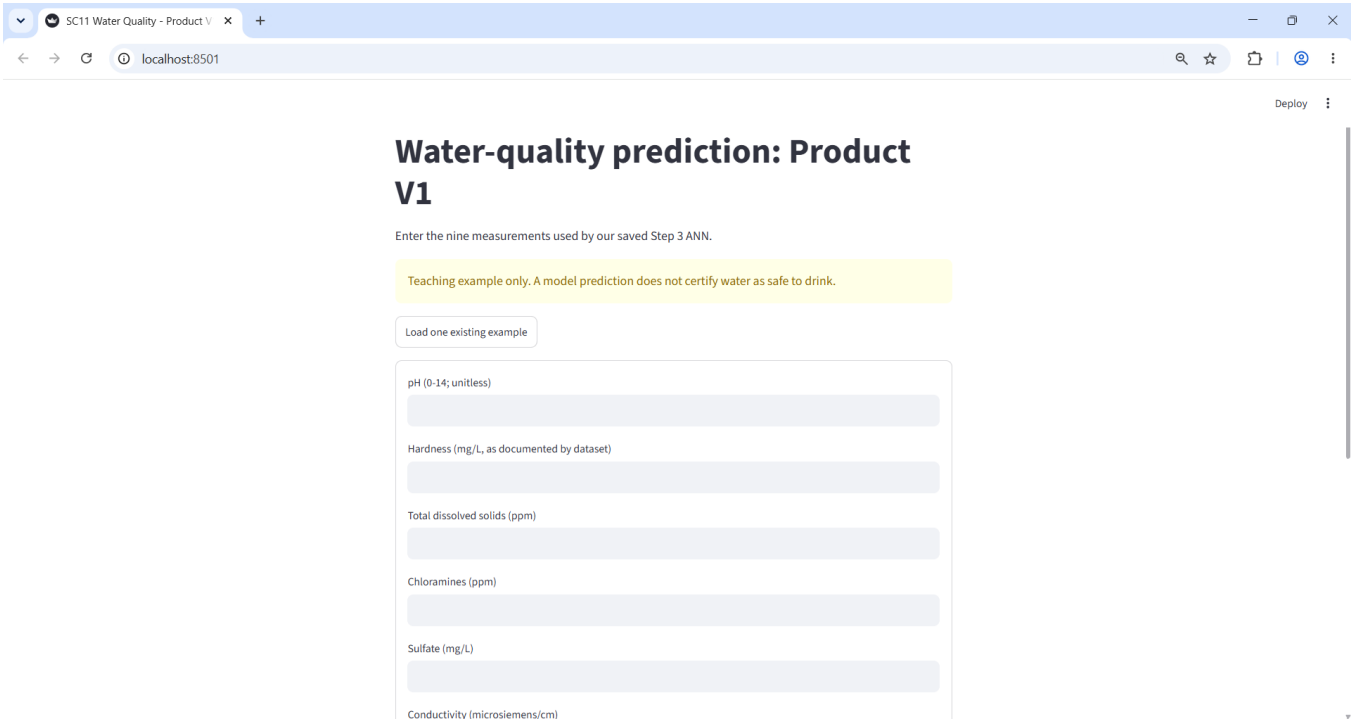
2. For SC11, wait for Streamlit's Local URL, usually **http://localhost:8501**, and open it. Other interfaces may show a different local address or application window.

Action 7 - illustration 2: The terminal shows the local address after Streamlit starts.

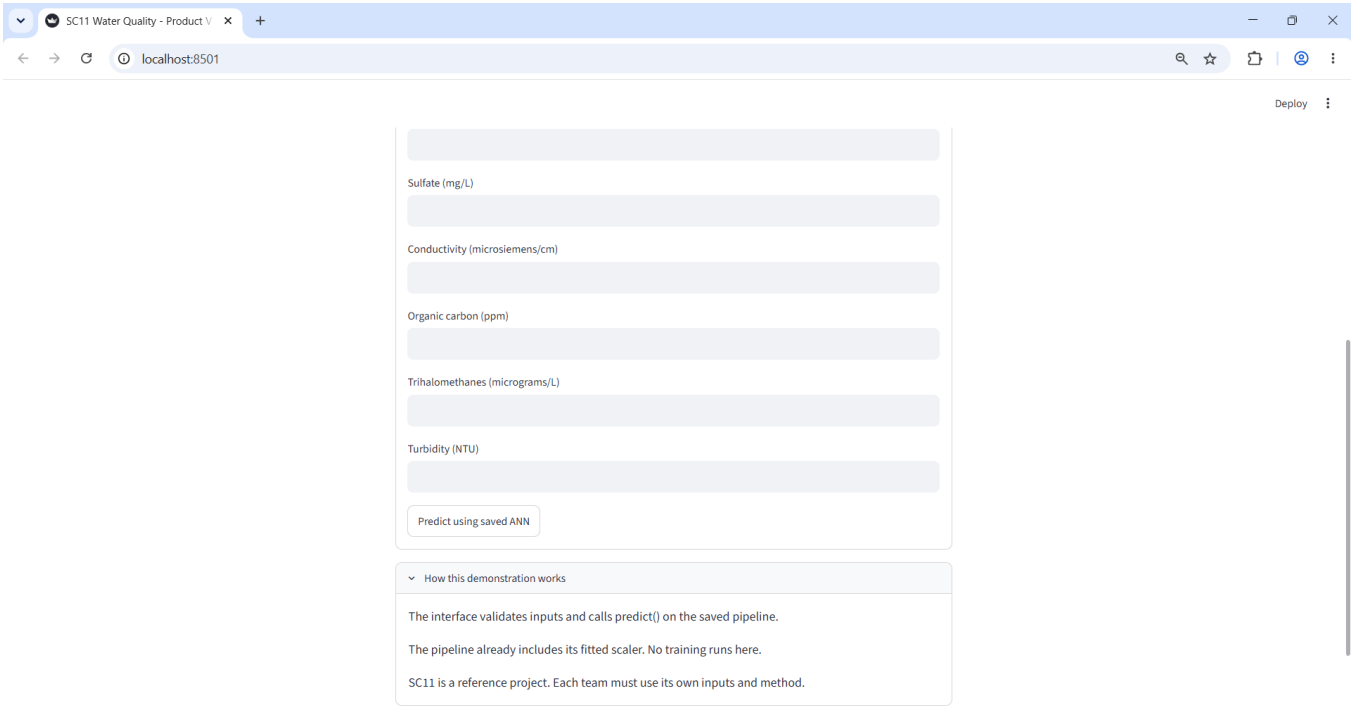


3. Check your project shows its title, purpose, relevant inputs and action button. SC11 has nine inputs. Resolve any model or controller loading error before continuing.

Action 7 - illustration 3: The browser displays the top of the Product V1 form and safety notice.



Action 7 - illustration 4: Scroll to the remaining inputs, prediction button and explanation.



4. Keep the terminal running while using the app. Stop it later with Ctrl+C. Local execution is sufficient for Step 5; hosting is optional.

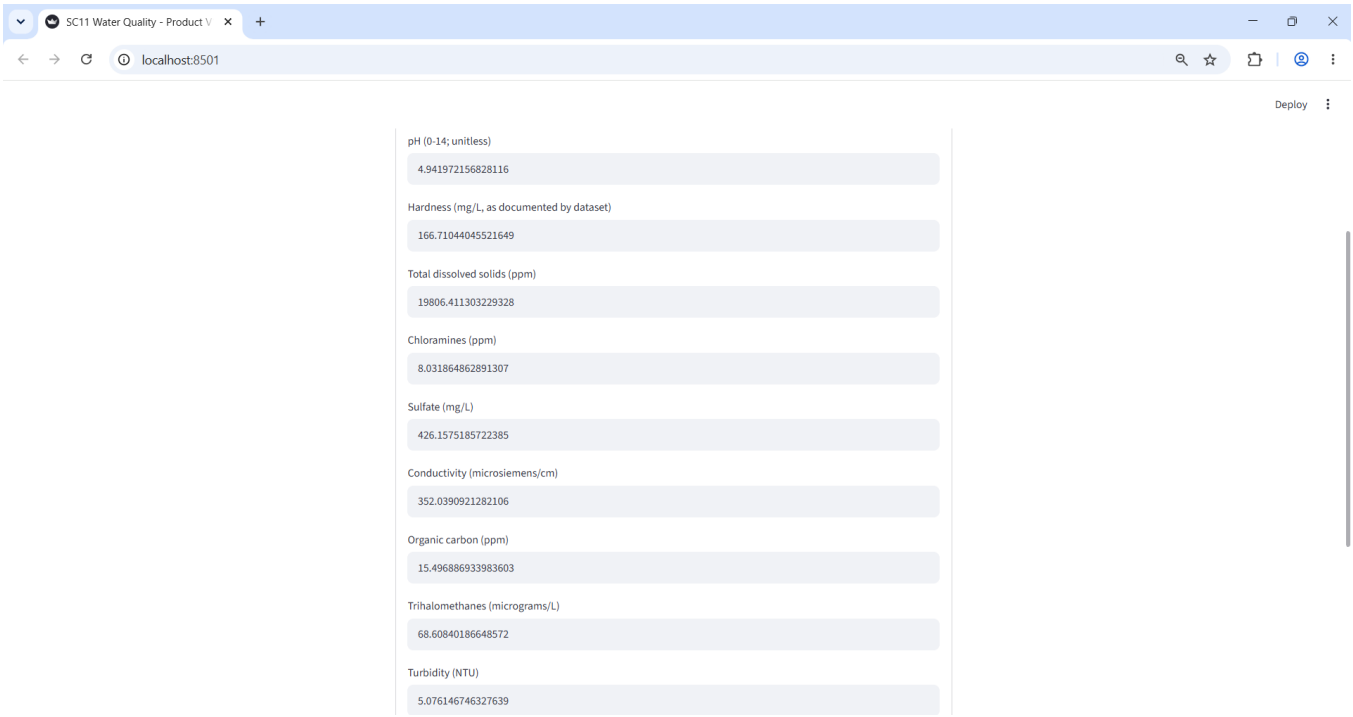
Action 8: Run a normal input

Why and how - A normal case: A normal case uses reasonable inputs to check that the complete path from form to result works. A displayed result alone does not prove the prediction is correct.

Expected evidence: A normal input produces an actual model output with a clear label.

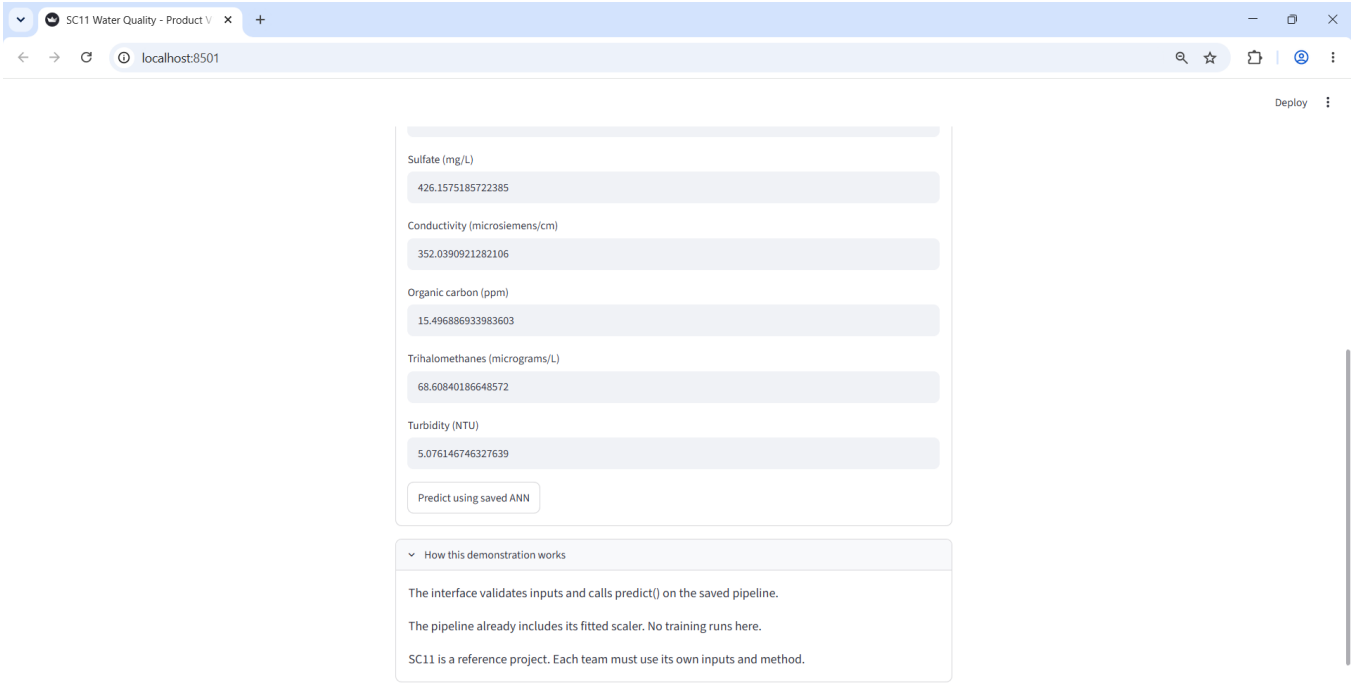
1. In the SC11 reference app, click **Load one existing example**. It reads the feature values from the first saved validation record, without using its label as the prediction.

Action 8 - illustration 1: Load the existing example to fill the input measurements.



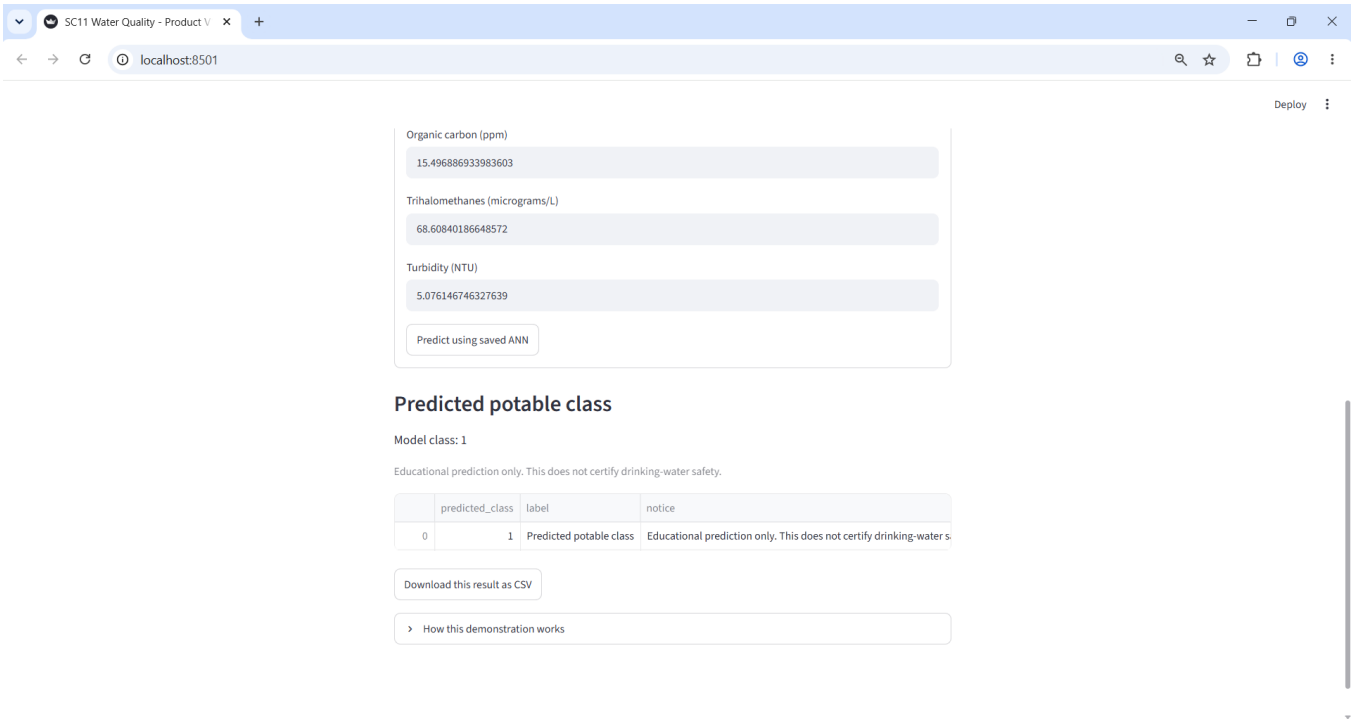
2. Read the populated values and units. Click **Predict using saved ANN**.

Action 8 - illustration 2: Review the remaining inputs and press the prediction button.



3. Check the predicted class, its plain-language label and the input/output table. Record what actually happened in the Step 5 test table.

Action 8 - illustration 3: The model returns a class and an educational-use notice for this example.



4. Do not claim a prediction is correct just because the app displayed one. Interface function and model quality are different checks; the earlier benchmark remains relevant.

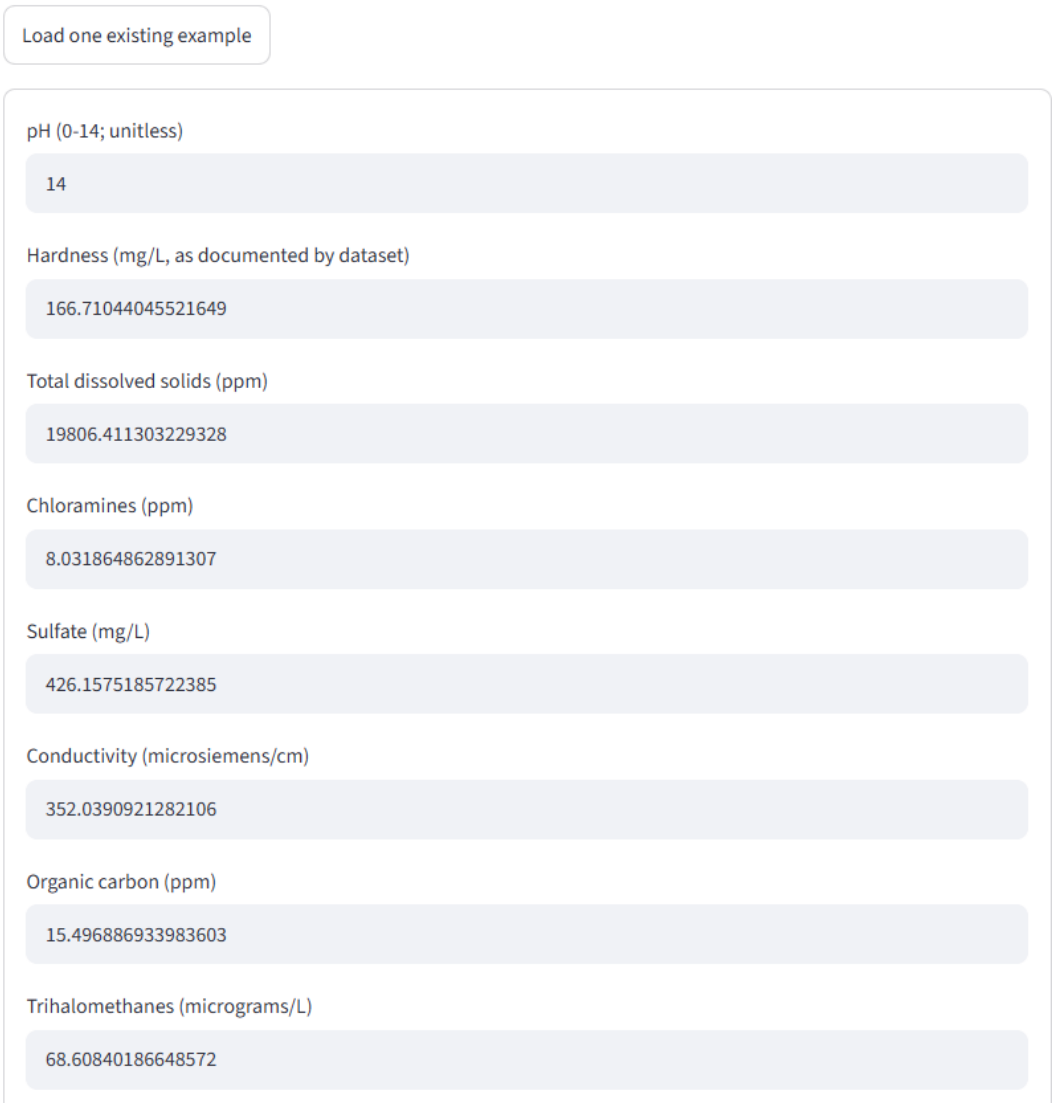
Action 9: Test boundaries and invalid input

Why and how - Boundary and invalid cases: A boundary is an edge of the accepted input range, such as pH 0 or 14 in this example. Invalid cases check that blanks, text and out-of-range values are rejected clearly. Other projects need their own rules.

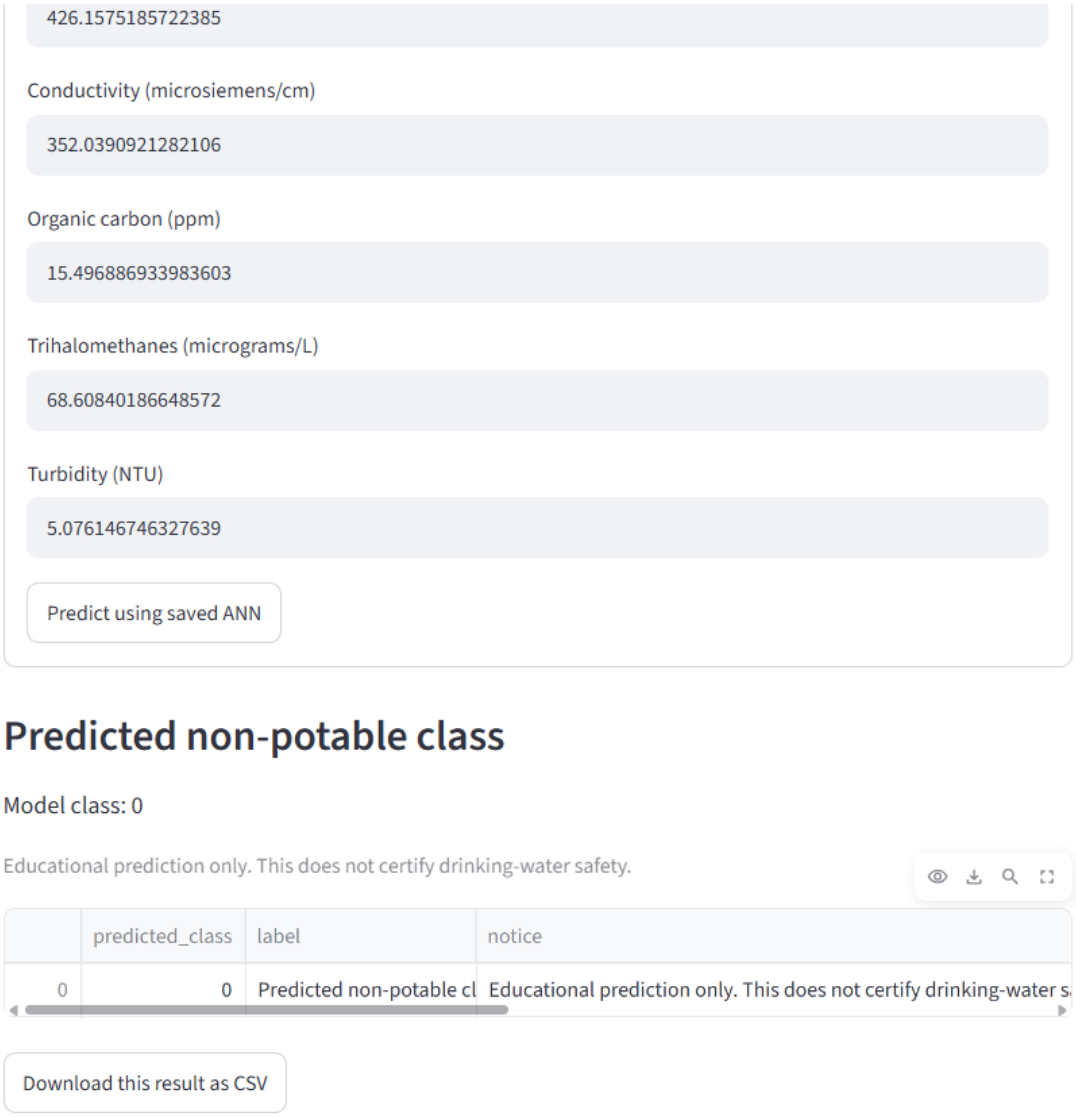
Expected evidence: The app explains an invalid input and recovers after correction.

1. Choose boundary cases for your own project. In SC11, set pH to 0 and then 14 in separate runs. These test validation limits, not recommended drinking-water values.

Action 9 - illustration 1: A boundary check sets pH to 14; this is not a recommended drinking-water value.

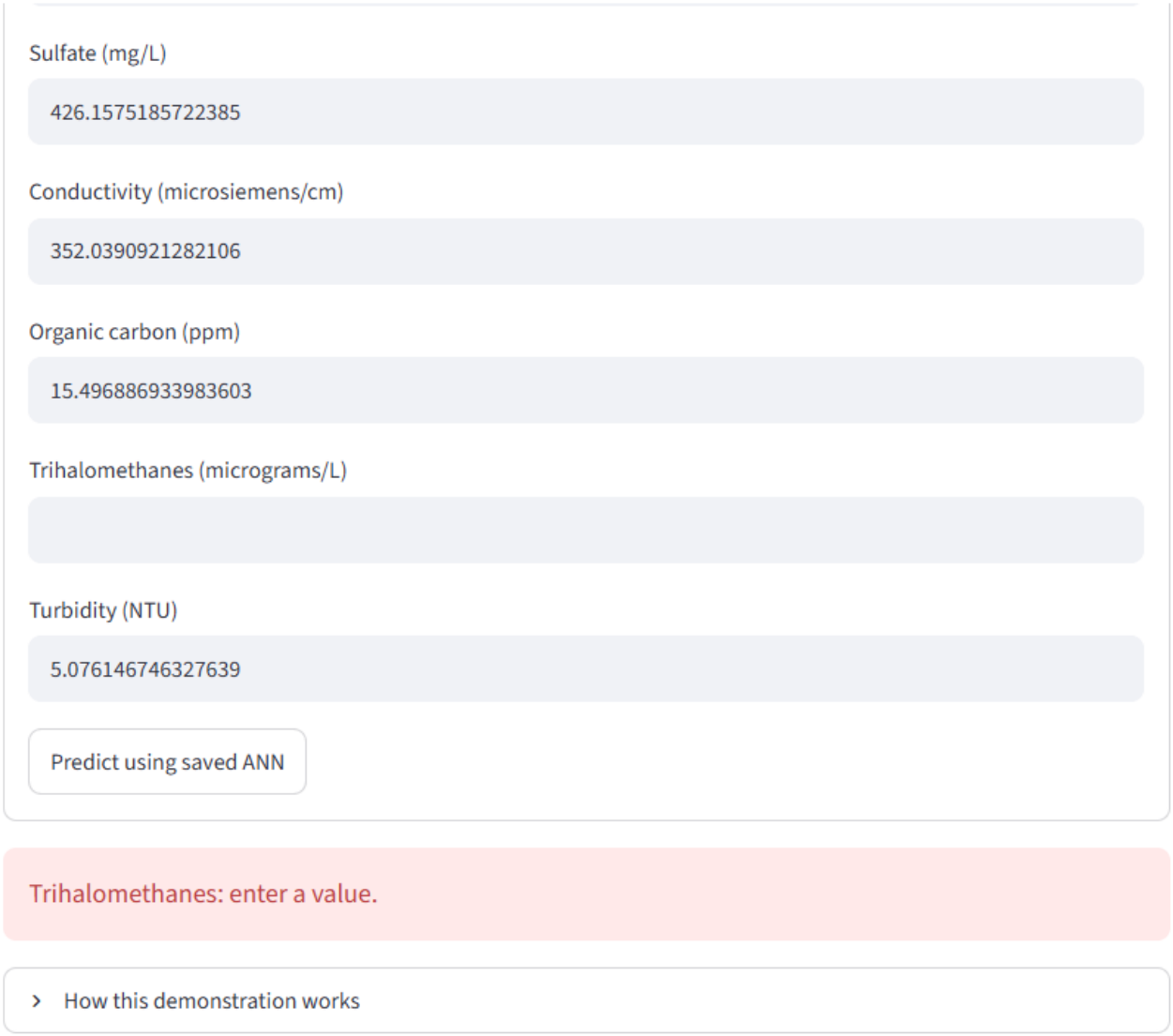


Action 9 - illustration 2: This boundary input produces a model result; record the actual observation.



2. Choose invalid cases for your project. SC11 examples include pH 15, a blank field, text such as abc, and negative turbidity. Test one change at a time.

Action 9 - illustration 3: A blank Trihalomethanes field produces a clear validation message.



3. Each invalid input should show a clear field-specific error, without an unhandled traceback or a new prediction from that input.

4. Record the tested input, expected response, observed response and Pass/Fail in docs/STEP-5.md. Restore a valid example before continuing.

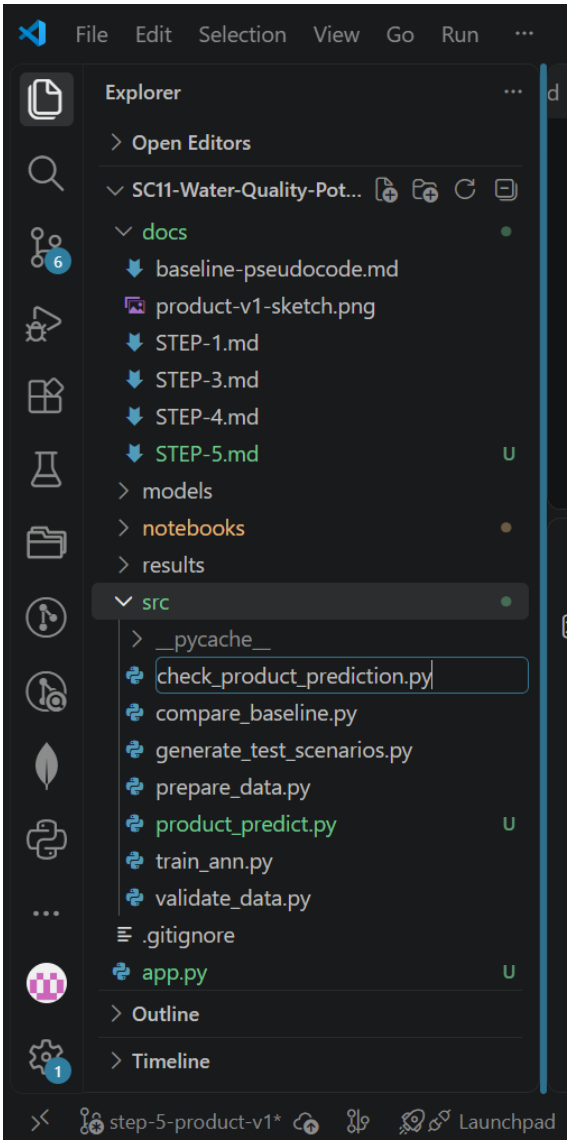
Action 10: Verify inference and result download

Why and how - Consistency check and CSV: CSV is a plain-text table. Compare the same inputs and the same saved method through the script and the interface: matching results help check that the connection is consistent, not that the model is accurate.

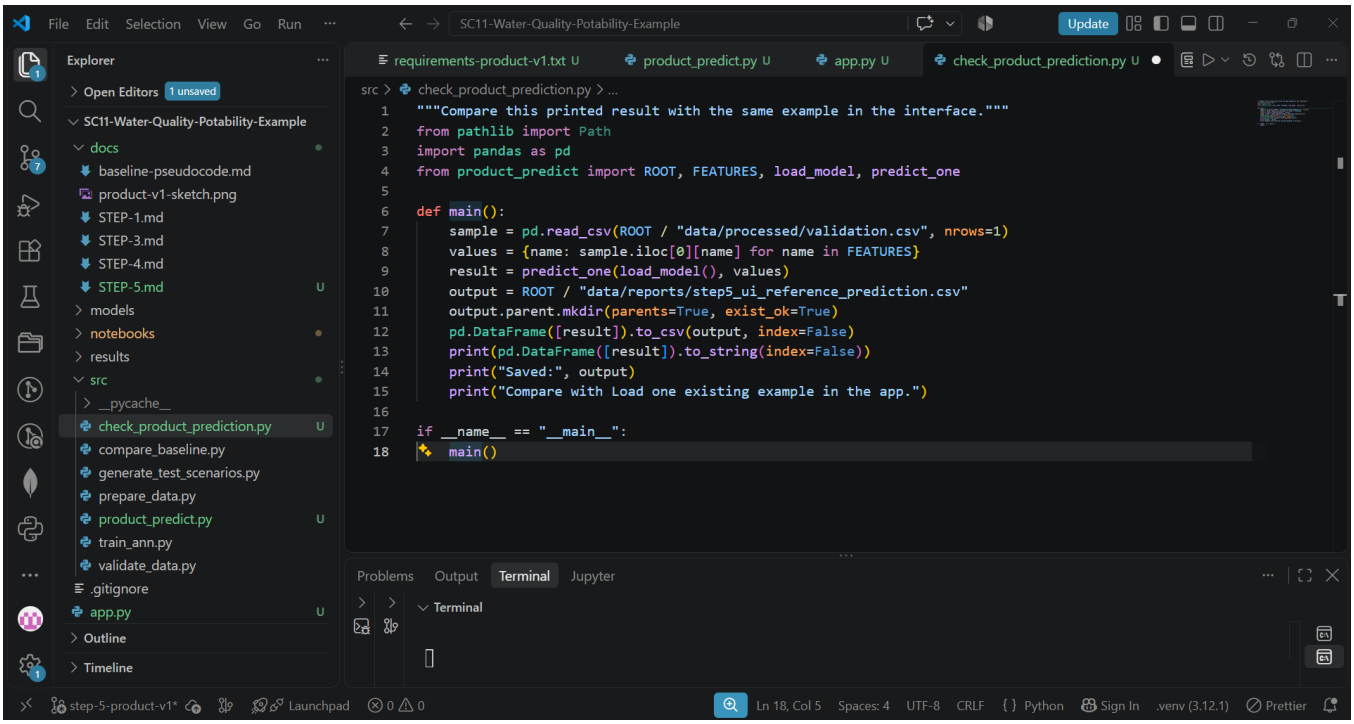
Expected evidence: The downloaded CSV matches the displayed input and output.

1. For your project, compare the same case through your interface and the underlying model, controller or optimiser. Appendix F illustrates this for SC11; its script and paths are specific to the water example.

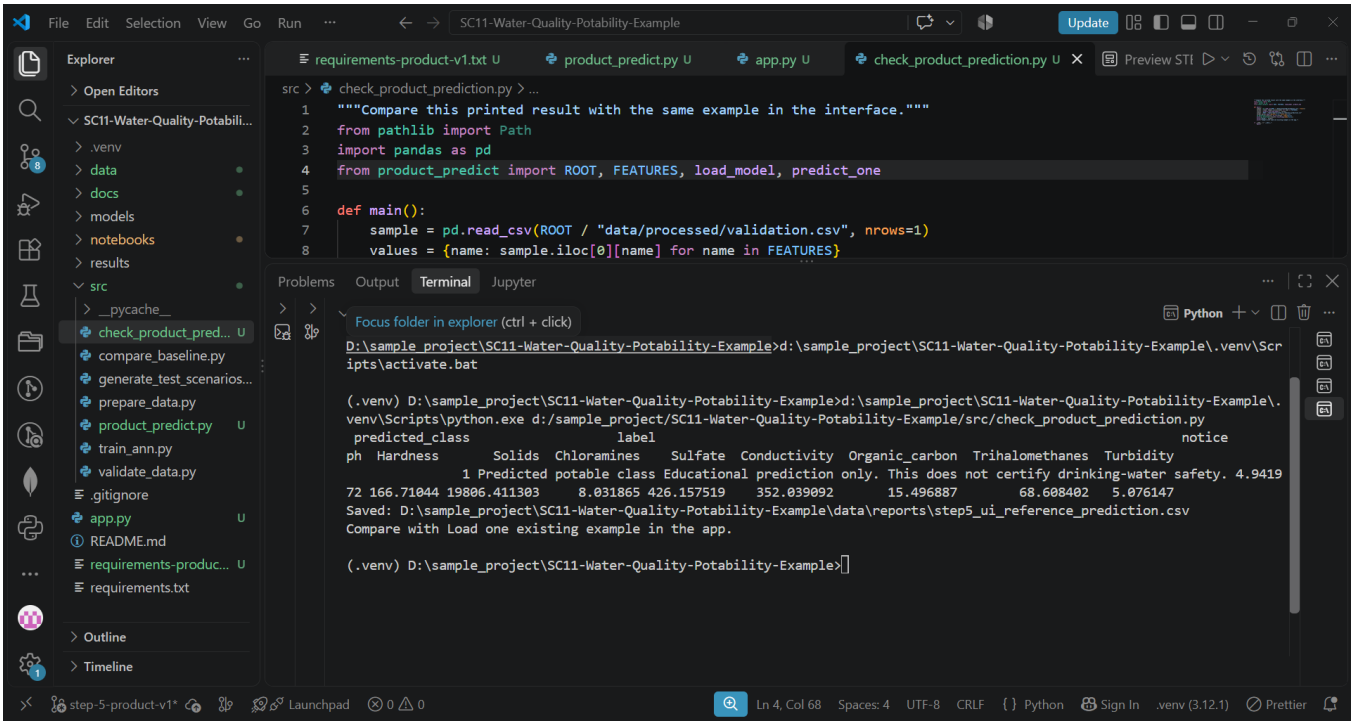
Action 10 - illustration 1: Create check_product_prediction.py inside src.



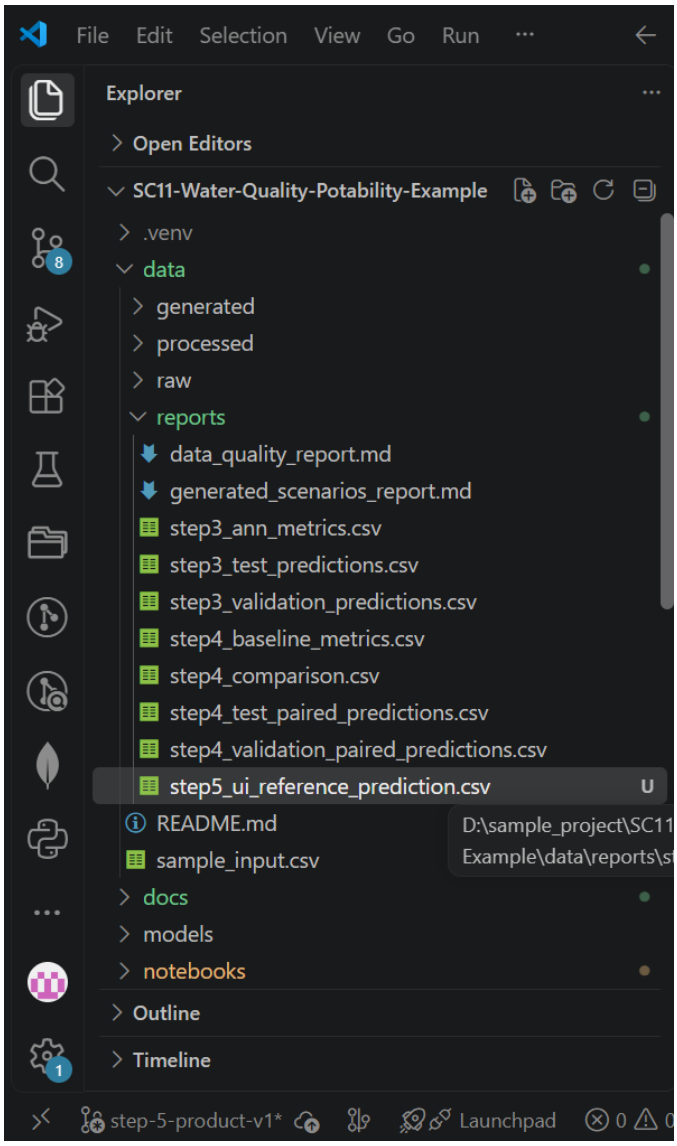
Action 10 - illustration 2: The comparison script uses the same saved model and validation example.



Action 10 - illustration 3: The terminal prints the script prediction and saved report path.

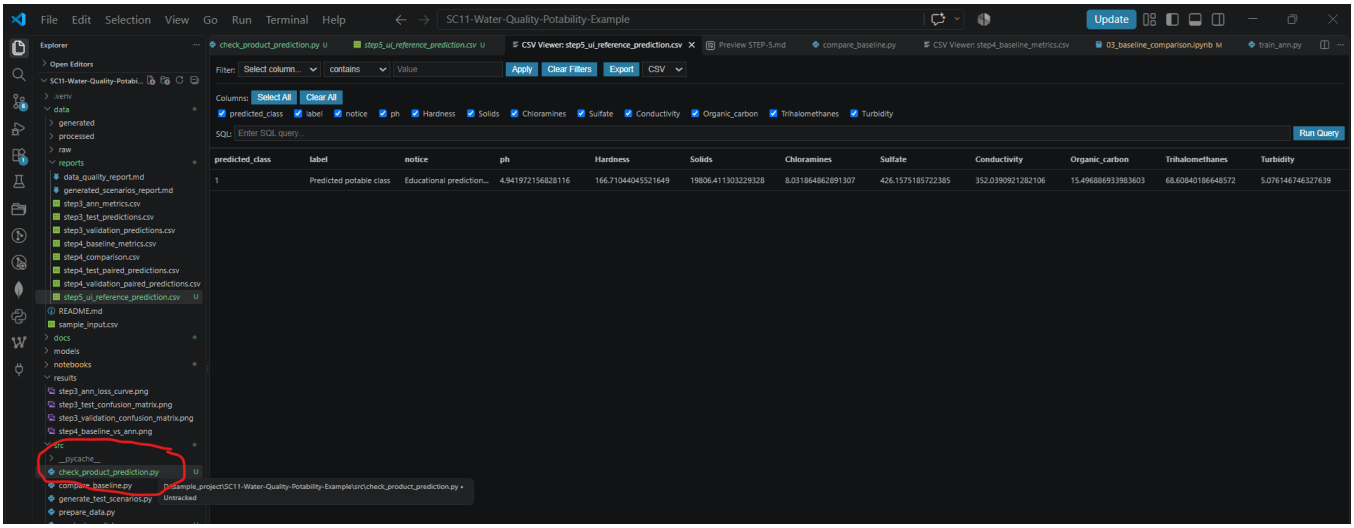


Action 10 - illustration 4: Find step5_ui_reference_prediction.csv in data/reports.

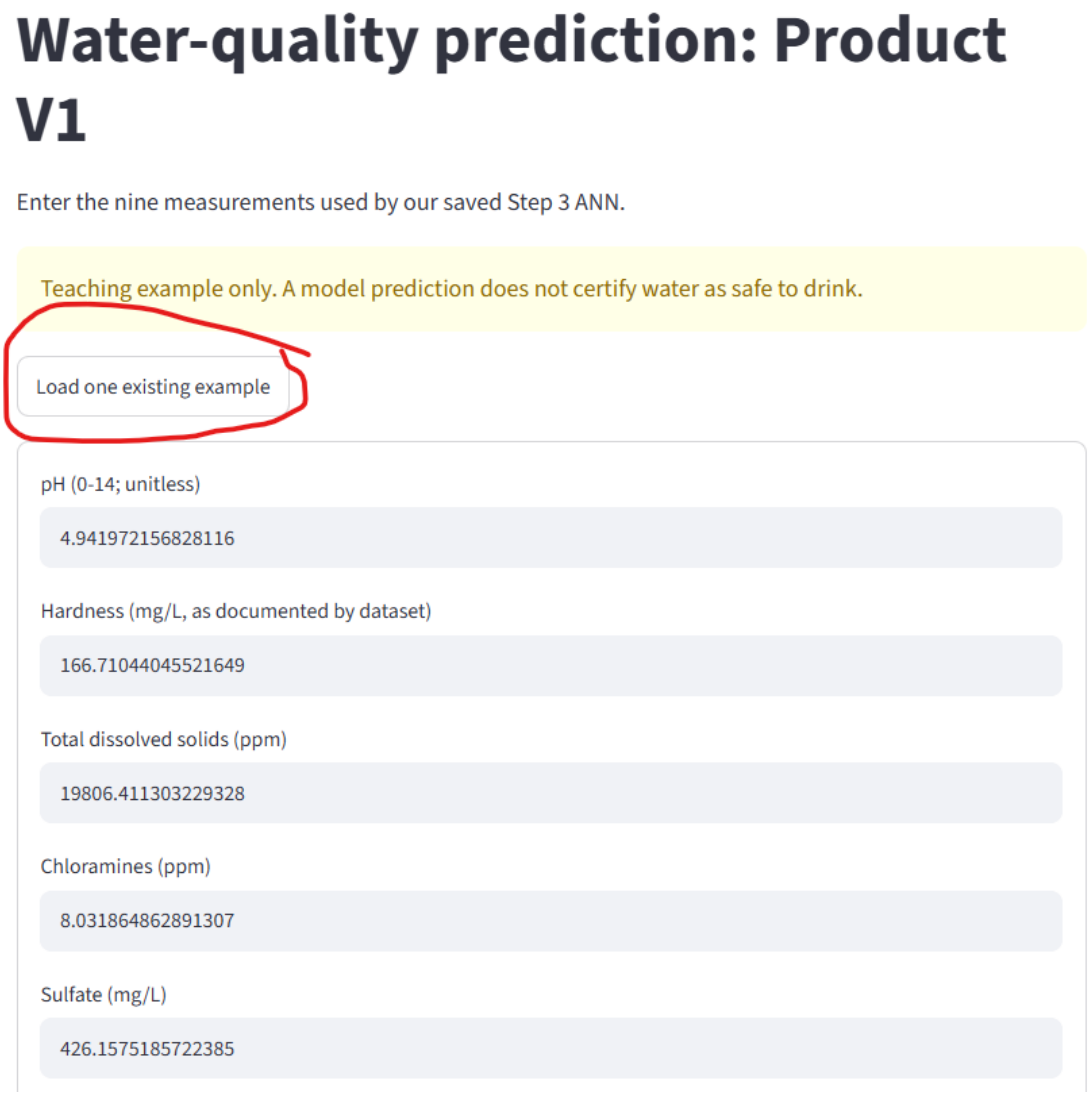


2. In SC11, load the same existing example in the interface and Predict; compare its input values and predicted class with the reference script. For your project compare the corresponding output for identical inputs and method.

Action 10 - illustration 5: Inspect the saved reference prediction in a CSV viewer.

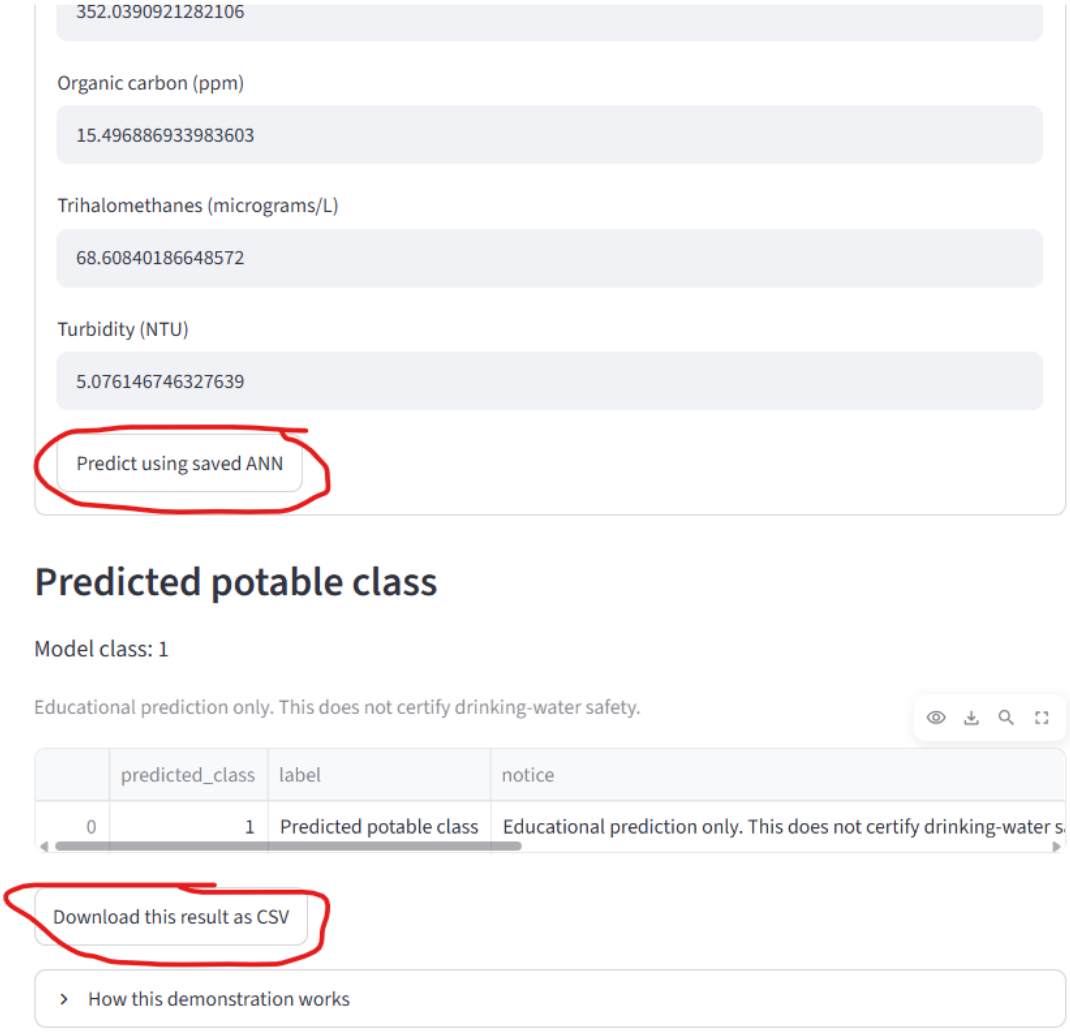


Action 10 - illustration 6: In the interface, load the same existing example.

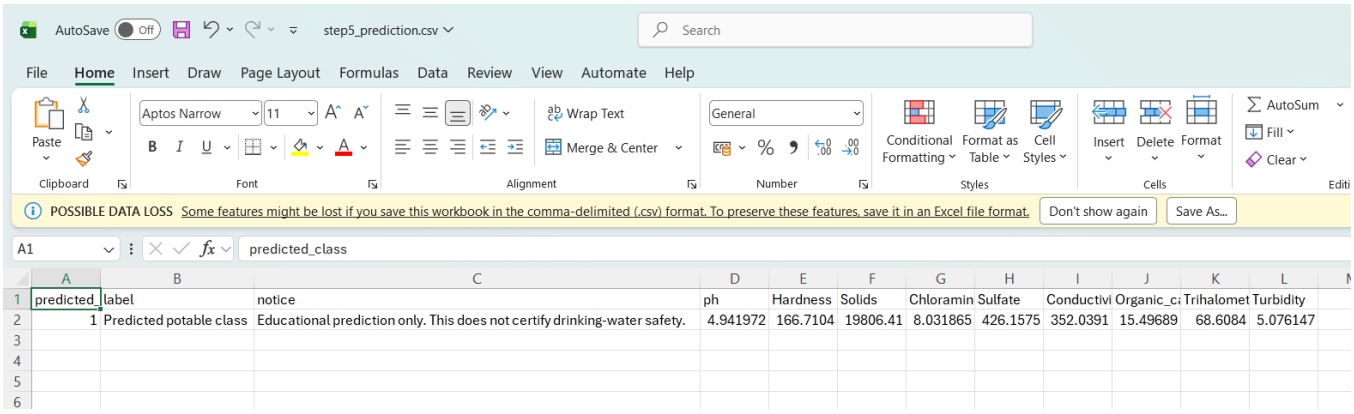


3. If your application offers a download, open it and check its actual contents. SC11's CSV contains input values, predicted class, label and limitation notice; your output may differ.

Action 10 - illustration 7: Predict and download the result as CSV.



Action 10 - illustration 8: Open the downloaded step5_prediction.csv and compare the values and class.



4. A CSV viewer is optional. If using it in VS Code, first open the CSV, then right-click its filename in Explorer and choose the installed CSV viewer command.
5. Keep the original benchmark results and the new interactive prediction separate. Do not overwrite the Step 3/4 evidence files.

Action 11: Run the automated checks

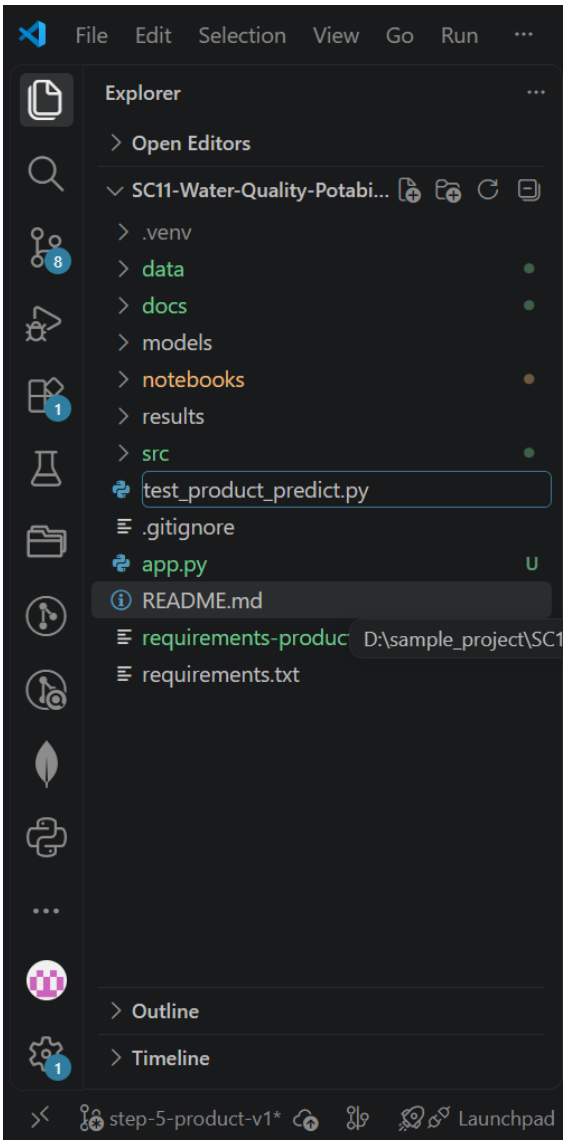
Why and how - Unit testing: A unit test automatically checks a small piece of program behaviour against an expected result. Repeat it after changes to catch mistakes early; test the full interface separately.

Expected evidence: Five validation tests pass, and manual model and interface checks are recorded.

1. **Why test?** Imagine entering a normal pH, then an impossible pH. We want the program to accept the first and reject the second. A unit test performs small checks like these automatically, so we can repeat them after changing code.

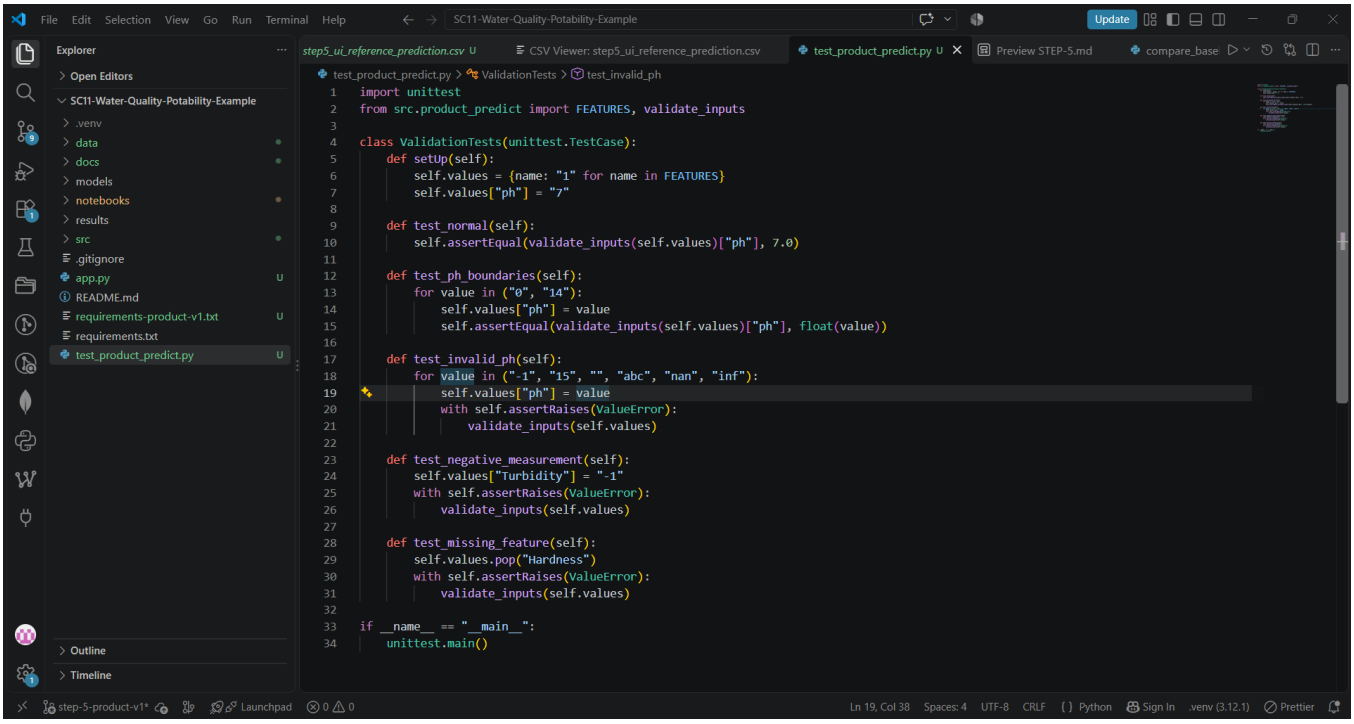
2. **What is unittest?** It is Python's built-in tool for automated checks; no separate unittest installation is needed. Appendix D shows SC11 tests in test_product_predict.py. Design tests for your own project's inputs and expected behaviour; other test tools are also possible.

Action 11 - illustration 1: Create the test file in the project root, as shown here.



3. In this example, **setUp** prepares a fresh valid input for each check. Methods beginning **test_** are the tests. **assertEqual** checks an expected value; **assertRaises(ValueError)** checks that invalid input is rejected.

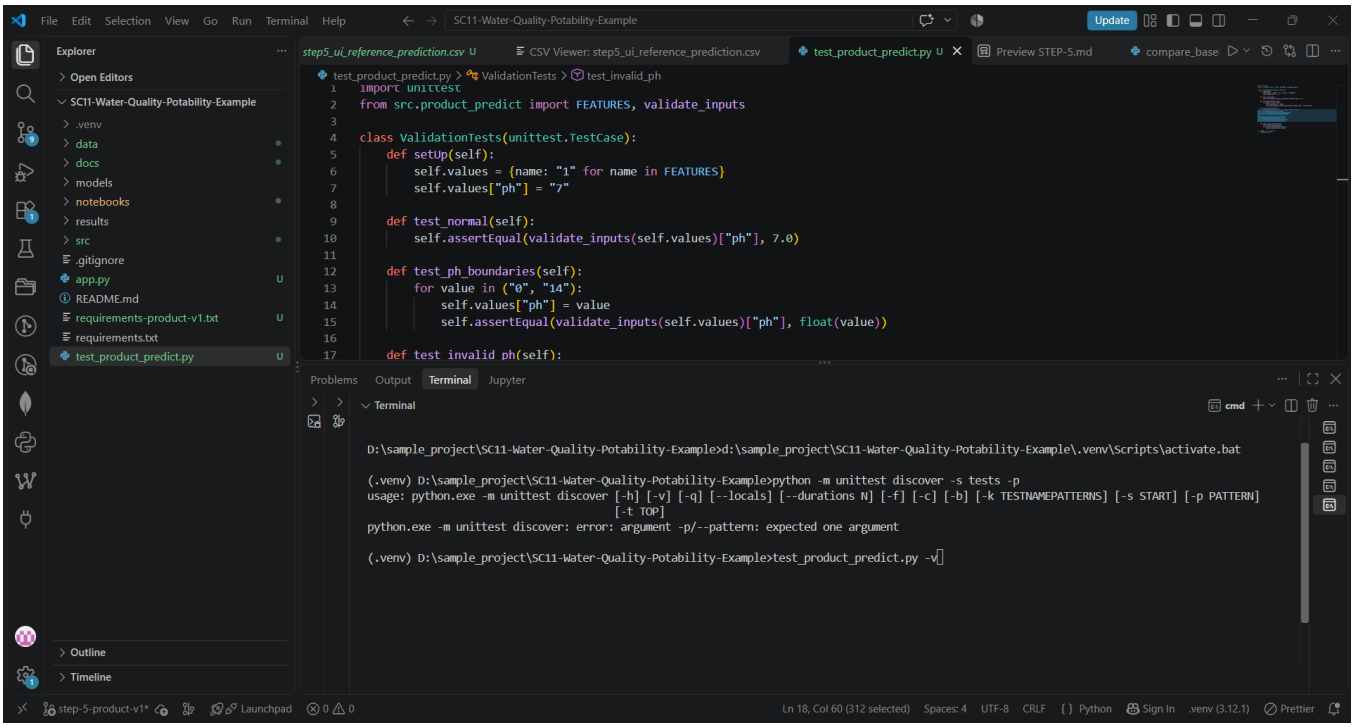
Action 11 - illustration 2: The SC11 example test file contains five test methods.



4. The five tests check: a normal input; pH limits 0 and 14; invalid pH values; a negative turbidity; and a missing Hardness input. Some methods try several values, but they still count as one test each.

5. **Understand the example error below:** Enter was pressed after **-p** before its required filename. The command was incomplete, so no tests ran. The separate filename typed at the next prompt is not part of that earlier command.

Action 11 - illustration 3: Example of an incomplete command: no tests ran. Use the complete corrected command below.

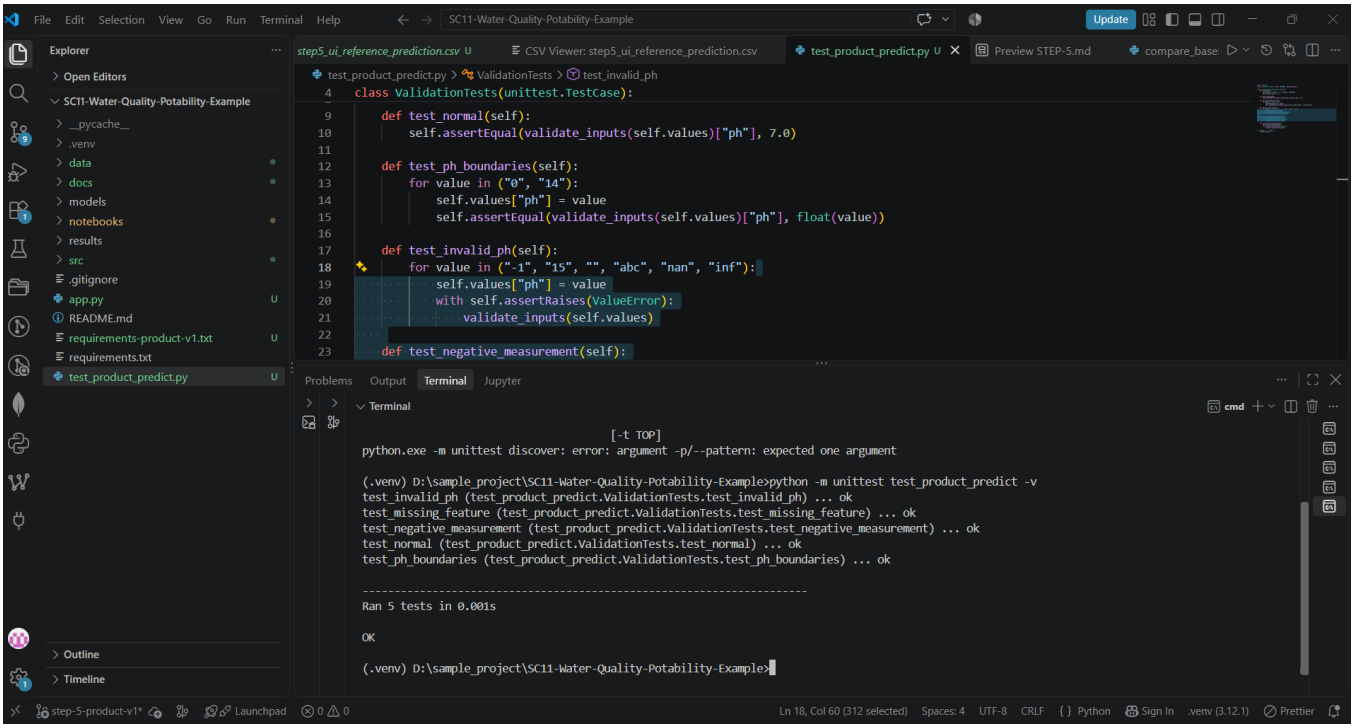


6. **Run the corrected command:** open the terminal in the project root, activate your project's environment, then type the entire line below before pressing Enter: **python -m unittest test_product_predict -v**

7. Here **python -m unittest** starts the test tool; **test_product_predict** identifies the file without .py; **-v** displays each test name. If the file is elsewhere, move it to the project root or use the correct command for its location.

8. **Read the result:** each test ending in **ok** passed; **Ran 5 tests** reports five test methods; final **OK** means all five passed. For invalid values, a pass means the expected error was correctly raised. If you see FAIL or ERROR, read the details and correct the cause; zero tests is not a successful check.

Action 11 - illustration 4: The corrected command ran all five tests. Each shows ok and the final result is OK. The earlier command error remains above it.



9. These tests check input validation only. They do not prove the model is accurate or that the interface and saved model work. Keep the separate manual checks in Actions 7-10. Use a temporary project copy to check the missing-model message; do not delete your original model.

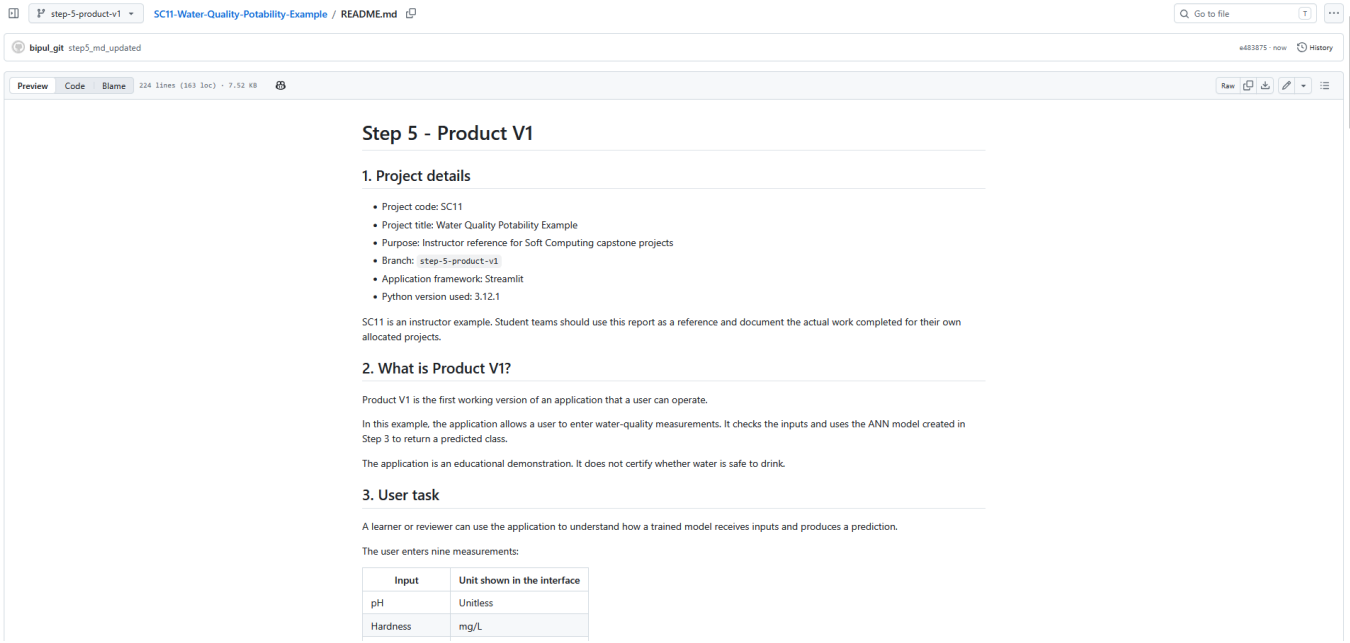
Action 12: Complete the report and evidence

Why and how - Evidence and reproducibility: A report helps another person run and understand your work. Record what you actually did, the observed results and limitations, so your reviewer can check the evidence.

Expected evidence: The finished report explains execution, observed tests and limitations.

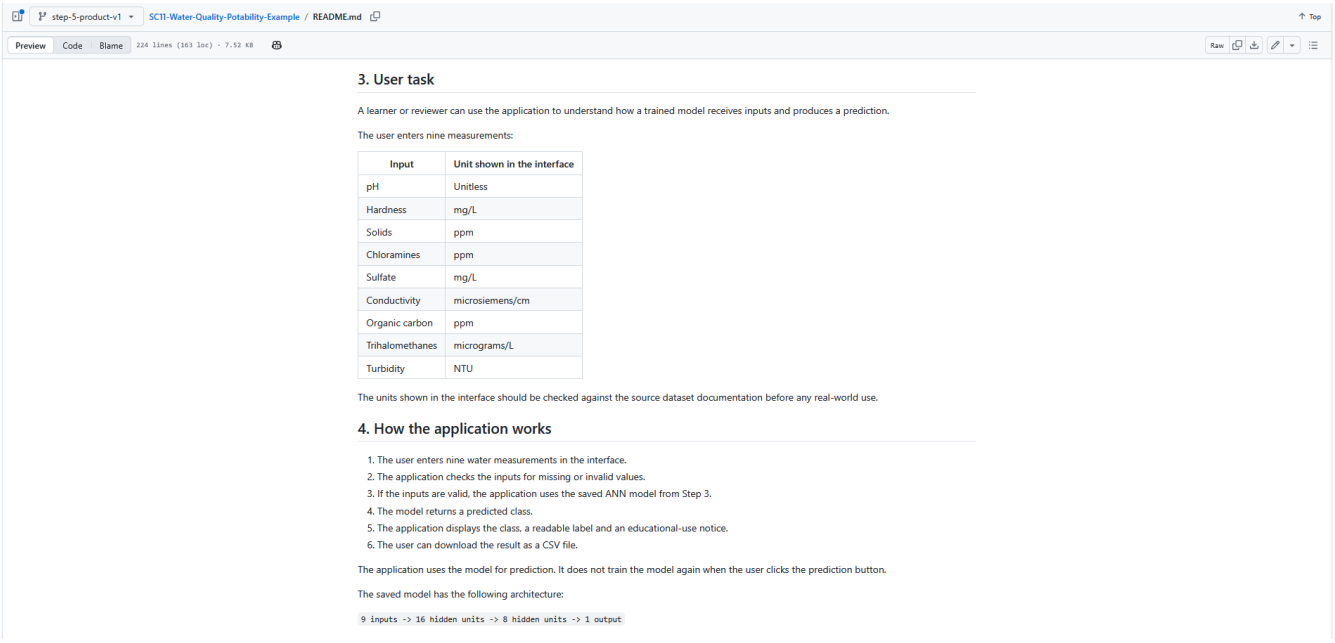
1. Write a report explaining your own Product V1. Appendix E offers optional headings; use your team's real project details, choices and observations. The images show an SC11 report rendered in README.md.

Action 12 - illustration 1: SC11 rendered report example: project details and purpose shown in README.md.



2. Record the entry file, launch command, Python/dependency versions, input/output definitions and model/controller path.

Action 12 - illustration 2: SC11 report example: inputs, units and the application flow.



step-5-product-v1 / SC11-Water-Quality-Potability-Example / README.md

Preview | Code | Blame | 224 lines (163 loc) · 7.52 KB

3. User task

A learner or reviewer can use the application to understand how a trained model receives inputs and produces a prediction.

The user enters nine measurements:

Input	Unit shown in the interface
pH	Unitless
Hardness	mg/L
Solids	ppm
Chloramines	ppm
Sulfate	mg/L
Conductivity	microsiemens/cm
Organic carbon	ppm
Trihalomethanes	micrograms/L
Turbidity	NTU

The units shown in the interface should be checked against the source dataset documentation before any real-world use.

4. How the application works

1. The user enters nine water measurements in the interface.
2. The application checks the inputs for missing or invalid values.
3. If the inputs are valid, the application uses the saved ANN model from Step 3.
4. The model returns a predicted class.
5. The application displays the class, a readable label and an educational-use notice.
6. The user can download the result as a CSV file.

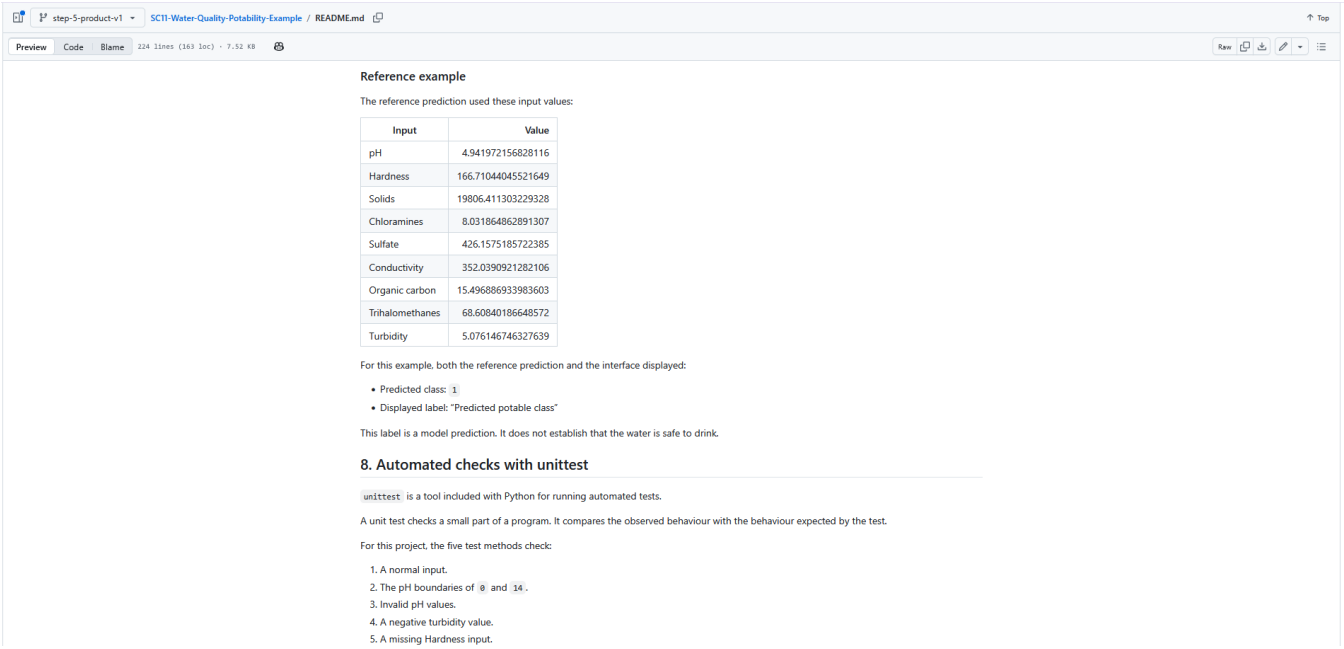
The application uses the model for prediction. It does not train the model again when the user clicks the prediction button.

The saved model has the following architecture:

```
9 inputs -> 16 hidden units -> 8 hidden units -> 1 output
```

3. Record actual tests and link to the files or results that support them; use screenshots when useful for showing the interface or an error. Describe feedback addressed and any issue still pending.

Action 12 - illustration 3: SC11 report example: measured prediction and explanation of automated checks.



step-5-product-v1 / SC11-Water-Quality-Potability-Example / README.md

Preview | Code | Blame | 224 lines (163 loc) · 7.52 KB

Reference example

The reference prediction used these input values:

Input	Value
pH	4.941972156828116
Hardness	166.71044045521649
Solids	19806.411303229328
Chloramines	8.031864862891307
Sulfate	426.1575185722385
Conductivity	352.0390921282106
Organic carbon	15.496886933983603
Trihalomethanes	68.60840186648572
Turbidity	5.076146746327639

For this example, both the reference prediction and the interface displayed:

- Predicted class: `1`
- Displayed label: "Predicted potable class"

This label is a model prediction. It does not establish that the water is safe to drink.

8. Automated checks with unittest

`unittest` is a tool included with Python for running automated tests.

A unit test checks a small part of a program. It compares the observed behaviour with the behaviour expected by the test.

For this project, the five test methods check:

1. A normal input.
2. The pH boundaries of `0` and `14`.
3. Invalid pH values.
4. A negative turbidity value.
5. A missing Hardness input.

4. Record each actual member's contribution. SC11 is an instructor example and must not claim student contributors.

5. Open Markdown Preview using Ctrl+Shift+V. Check the report is readable and links work. No claim of a successful test should be based on an unexecuted template.

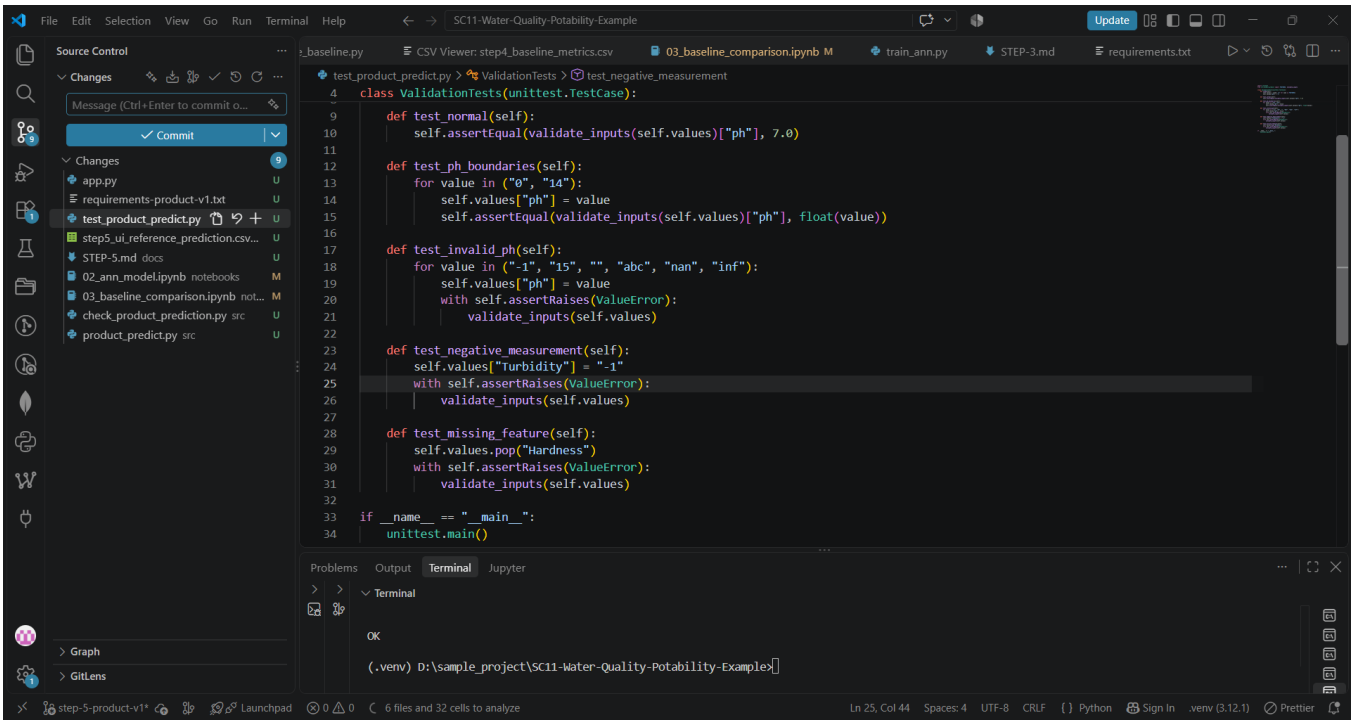
Action 13: Review, stage and commit

Why and how - Staging and commit: Staging selects the changes to include. A commit records a named snapshot in your local repository. Review the list first so the snapshot contains the intended work.

Expected evidence: Only the intended Product V1 files are staged with a meaningful message.

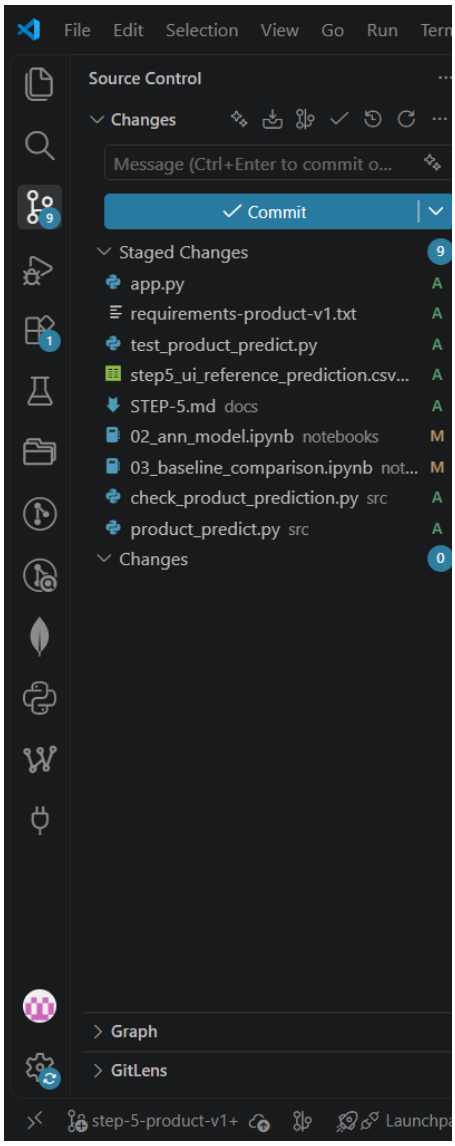
1. Save all intended files. Open Source Control and inspect each changed file before staging.

Action 13 - illustration 1: Source Control lists changed files; review each before staging.



2. Stage your project's interface, method, dependencies, tests, completed report and actual evidence. Review modified notebooks as well; stage only relevant changes. SC11's filenames are examples.

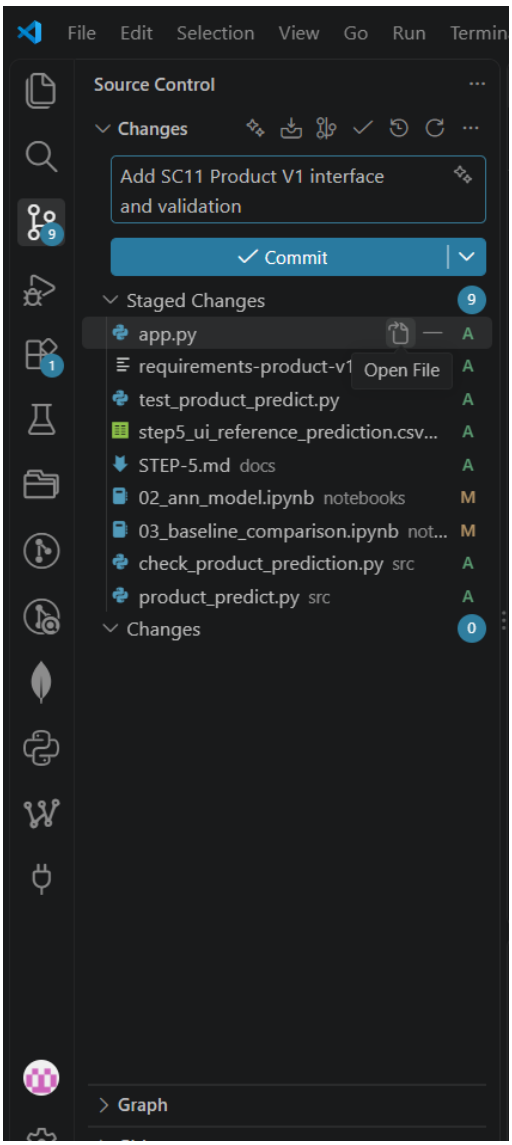
Action 13 - illustration 2: The example shows files staged for the next commit; inspect modified notebooks as well.



3. Do not stage .venv, caches, access codes or personal information. Check that earlier Step 3/4 evidence is still present.

4. Enter a meaningful commit message such as **Add SC11 Product V1 interface and validation**, then click Commit.

Action 13 - illustration 3: A meaningful commit message describes the Product V1 work.



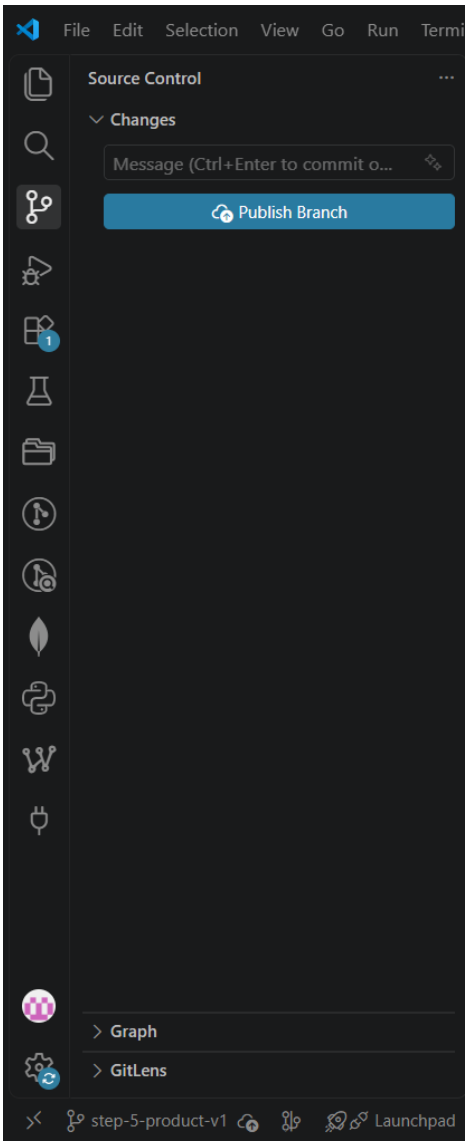
Action 14: Publish and verify the branch

Why and how - Publish and verify: Publishing or pushing sends local commits to the remote GitHub repository. Open the branch on GitHub afterwards to confirm the intended files reached it.

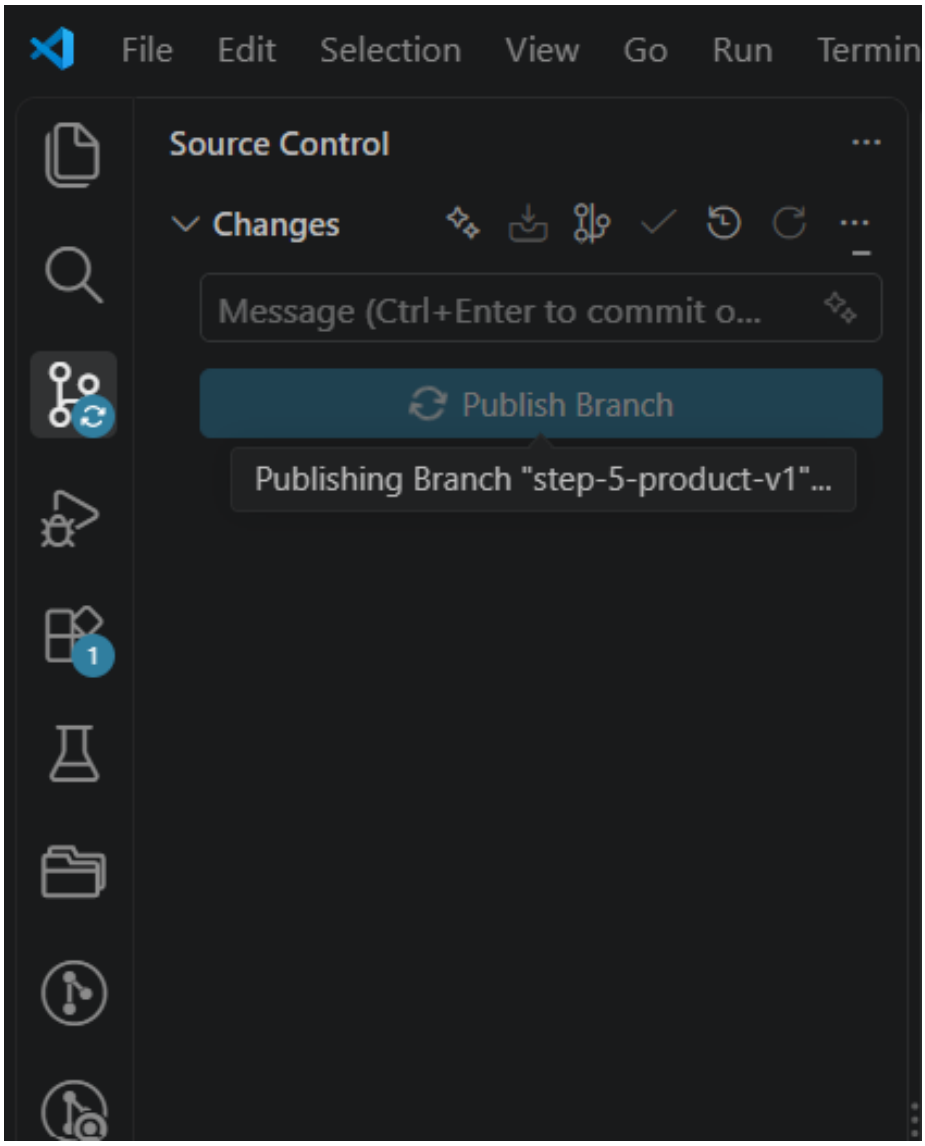
Expected evidence: GitHub shows the new branch, commit and Product V1 files.

1. Click **Publish Branch** or **Sync Changes** in VS Code. Wait for the operation to finish.

Action 14 - illustration 1: After the local commit, Publish Branch is available.

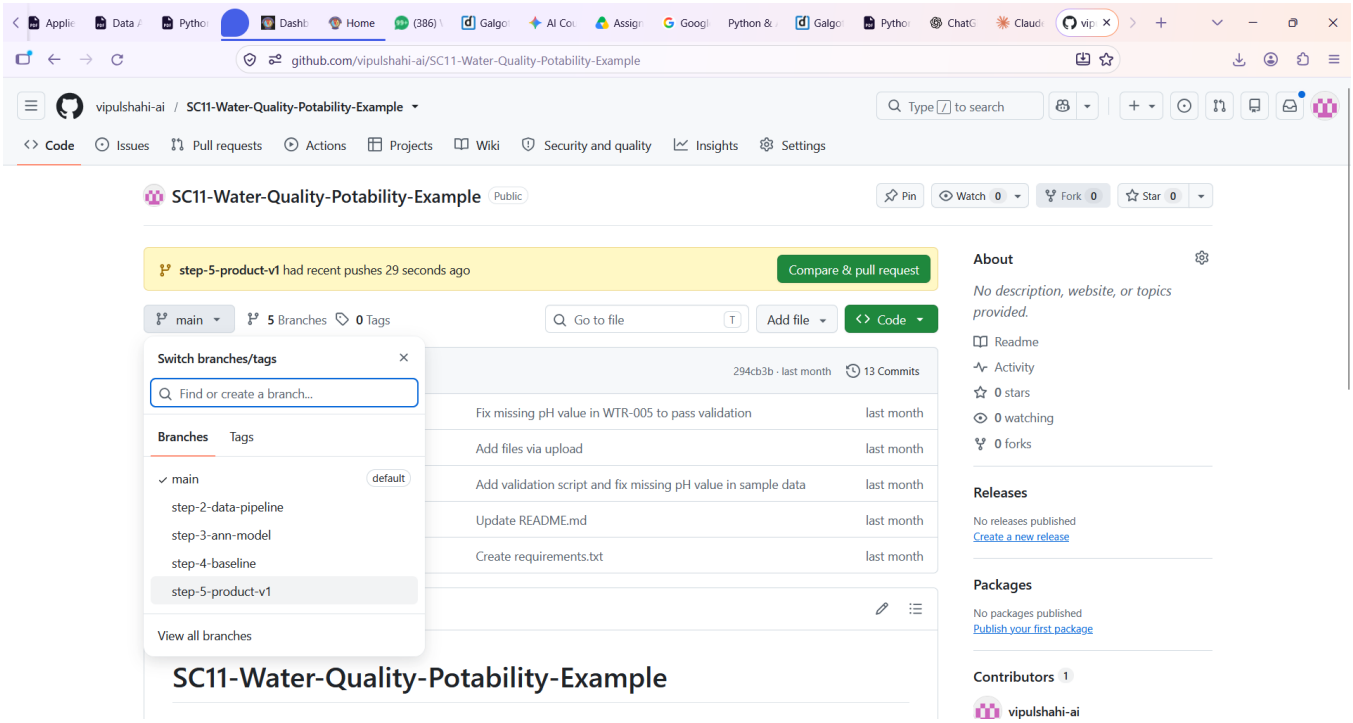


Action 14 - illustration 2: Publishing is in progress; wait for it to finish.



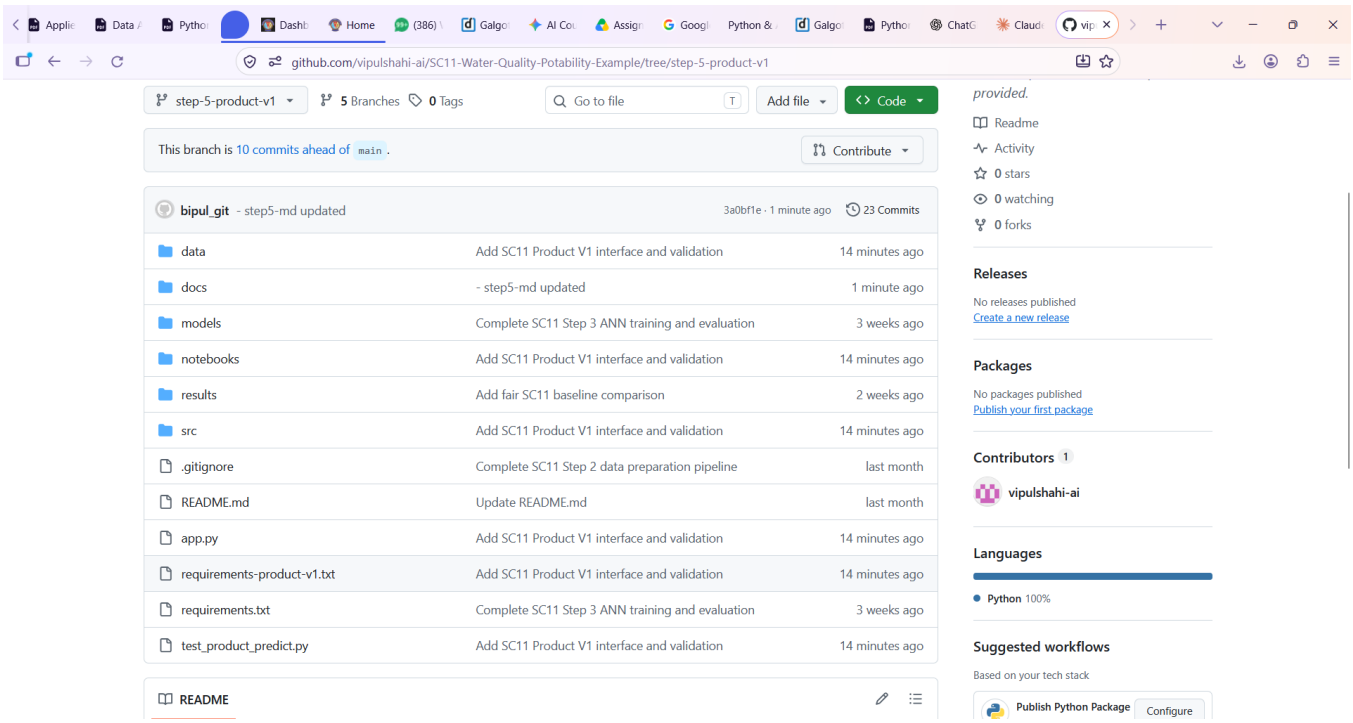
2. Open your repository in GitHub and select your Step 5 branch. The SC11 example uses **step-5-product-v1**.

Action 14 - illustration 3: On GitHub, open the branch selector and select the Step 5 branch.

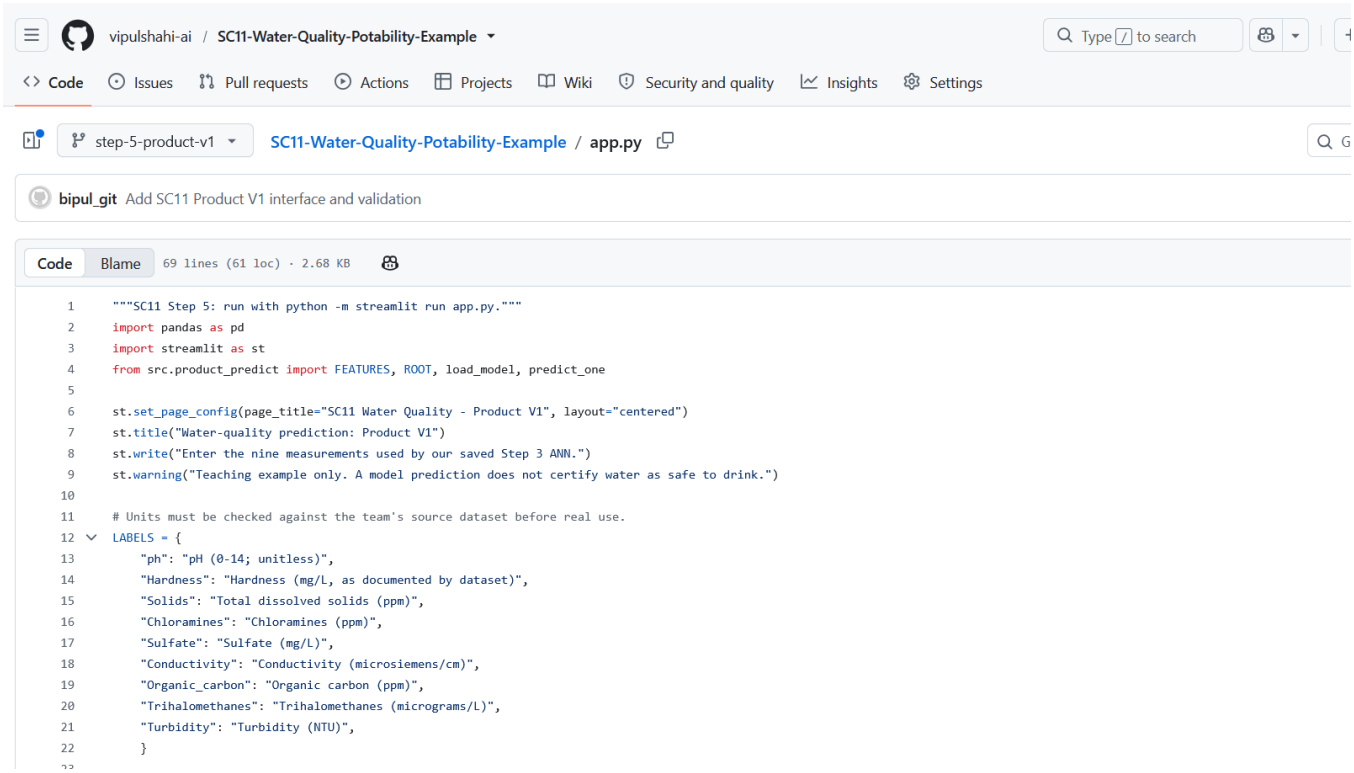


3. Check the latest commit and open your interface, method and report files. Confirm the online files match your local work.

Action 14 - illustration 4: The GitHub page shows the selected branch and its files.



Action 14 - illustration 5: Open an uploaded file to check its contents on the selected branch.



4. If the latest files are missing, finish the push or resolve the Git error first. A local commit or a publishing-in-progress screen is not proof that the files reached GitHub.

Action 15: Submit Step 5 on the project portal

Why and how - Submission and review: The portal links your team and milestone to the repository evidence. Check the branch and report paths and wait for a success status so faculty can locate the work.

- **1.** An allocated team's leader opens the dashboard with the admission number and access code already provided privately. Select **Step 5** after submitting Step 4.
- **2.** Enter the public repository URL and the actual branch name. Select the interface type and enter the entry file, launch command and Step 5 report path.
- **3.** Leave the hosted-demo link blank if your team is demonstrating locally. Confirm the four evidence checks only after completing them, and add a brief completion note.
- **4.** Submit and wait for the success message. Check the Step 5 timeline status. Later revisions keep their own recorded versions; the first submission determines timeliness.
- **5.** SC11 is an instructor reference, not an allocated student team. Do not submit this example through a student's dashboard. Keep access codes out of

screenshots.

Expected evidence: An allocated team sees a successful Step 5 submission and status.

Troubleshooting

- Model missing: check the branch and restore the Step 3 model and metadata in models/.
- Unpickling or version error: use the original training environment and confirm the saved model file is valid. Do not conceal a loading error with a fake result.
- Module not found: select the correct interpreter and install dependencies in that environment.
- Wrong feature count/order: compare the app schema with the actual model metadata; do not reorder values by guesswork.
- Port already in use: stop the older app with Ctrl+C or use --server.port 8502.
- Portal Step 5 blocked: submit Step 4 first with the same allocated team.
- A test passes but the UI fails: record the failure and fix the interface; automated validation checks cover only part of the product.

How to use the worked examples

- A - Dependency example: identify packages required by your own framework and method; record their versions and setup. Streamlit and SC11 requirements are examples.
- B - Method connection: study validation and loading. For an irrigation model use your soil and weather inputs; for a scheduler call its own scheduling logic and constraints.
- C - Interface example: examine how a form calls the method and displays a response. Replace water fields, labels, validation and output with the needs of your own user.
- D - Test example: choose normal, boundary and invalid cases for your own rules. unittest is one option; the SC11 tests and pH boundaries are not universal.
- E - Report outline: reuse suitable headings, but write your own project, method, actual observations, limitations and contributions. The brackets are prompts, not results.
- F - Consistency check: compare the same input through your application and its underlying method. Adapt the check to your own data and expected output.

Teams can write their own code or adapt suitable parts after understanding them. Run and test the resulting solution, and document what your team actually built.

Appendix A: requirements-product-v1.txt

SC11 worked example for learning: inspect its structure and purpose, then design or adapt a solution for your own project. The water fields, model, filenames and results are not requirements for other projects.

```
-r requirements.txt
streamlit
```

Appendix B: src/product_predict.py

SC11 worked example for learning: inspect its structure and purpose, then design or adapt a solution for your own project. The water fields, model, filenames and results are not requirements for other projects.

```
"""SC11 Product V1 inference. Adapt the schema for your own project."""
from pathlib import Path
import json
import math
import pickle

FEATURES = [
    "ph", "Hardness", "Solids", "Chloramines", "Sulfate",
    "Conductivity", "Organic_carbon", "Trihalomethanes", "Turbidity",
]
ROOT = Path(__file__).resolve().parents[1]
MODEL_PATH = ROOT / "models" / "ann_water_potability.pkl"
META_PATH = ROOT / "models" / "ann_model_metadata.json"


def validate_inputs(values):
    """Reject blank, nonnumeric, nonfinite and physically invalid values."""
    clean = {}
    for name in FEATURES:
        raw = values.get(name)
        if raw is None or str(raw).strip() == "":
            raise ValueError(f"{name}: enter a value.")
        try:
            number = float(raw)
        except (ValueError, TypeError) as error:
            raise ValueError(f"{name}: enter a number.") from error
        if not math.isfinite(number):
            raise ValueError(f"{name}: use a finite number.")
        if name == "ph" and not 0 <= number <= 14:
            raise ValueError("ph: use a value from 0 to 14.")
        if name != "ph" and number < 0:
            raise ValueError(f"{name}: negative values are invalid.")
        clean[name] = number
    return clean
```

Appendix B: src/product_predict.py (continued)

```
def load_model():
    """Load only the model you created and trust in this repository."""
    if not MODEL_PATH.is_file() or not META_PATH.is_file():
        raise FileNotFoundError(
            "Saved model or metadata is missing. Restore your Step 3 files."
        )
    metadata = json.loads(META_PATH.read_text(encoding="utf-8"))
    if metadata.get("features") != FEATURES:
        raise ValueError("Model metadata feature order does not match the app.")
    with MODEL_PATH.open("rb") as source:
        model = pickle.load(source)
    if list(getattr(model, "feature_names_in_", [])) != FEATURES:
        raise ValueError("Saved model input columns do not match the app.")
    return model


def predict_one(model, values):
    import pandas as pd

    clean = validate_inputs(values)
    frame = pd.DataFrame([[clean[name] for name in FEATURES]], columns=FEATURES)
    prediction = int(model.predict(frame)[0])
    if prediction not in (0, 1):
        raise ValueError("Unexpected model class. Check the saved model.")
    return {
        "predicted_class": prediction,
        "label": "Predicted potable class" if prediction == 1
        else "Predicted non-potable class",
        "notice": "Educational prediction only. This does not certify drinking-water safety.",
        **clean,
    }
```

Appendix C: app.py

SC11 worked example for learning: inspect its structure and purpose, then design or adapt a solution for your own project. The water fields, model, filenames and results are not requirements for other projects.

```
"""SC11 Step 5: run with python -m streamlit run app.py."""
import pandas as pd
import streamlit as st
from src.product_predict import FEATURES, ROOT, load_model, predict_one

st.set_page_config(page_title="SC11 Water Quality - Product V1", layout="centered")
st.title("Water-quality prediction: Product V1")
st.write("Enter the nine measurements used by our saved Step 3 ANN.")
st.warning("Teaching example only. A model prediction does not certify water as safe to drink.")

# Units must be checked against the team's source dataset before real use.
LABELS = {
    "ph": "pH (0-14; unitless)",
    "Hardness": "Hardness (mg/L, as documented by dataset)",
    "Solids": "Total dissolved solids (ppm)",
    "Chloramines": "Chloramines (ppm)",
    "Sulfate": "Sulfate (mg/L)",
    "Conductivity": "Conductivity (microsiemens/cm)",
    "Organic_carbon": "Organic carbon (ppm)",
    "Trihalomethanes": "Trihalomethanes (micrograms/L)",
    "Turbidity": "Turbidity (NTU)",
}


@st.cache_resource
def saved_model():
    return load_model()


try:
    model = saved_model()
except Exception as error:
    st.error(f"The saved model could not be loaded: {error}")
    st.info("Use the Step 3 Python environment and check the model and metadata paths.")
    st.stop()
```

Appendix C: app.py (continued)

```
if st.button("Load one existing example"):
    try:
        example = pd.read_csv(ROOT / "data" / "processed" / "validation.csv", nrows=1)
        for name in FEATURES:
            st.session_state[name] = str(example.iloc[0][name])
    except Exception as error:
        st.error(f"Example could not be loaded: {error}")

with st.form("water_measurements"):
    values = {name: st.text_input(LABELS[name], key=name) for name in FEATURES}
    submitted = st.form_submit_button("Predict using saved ANN")

if submitted:
    try:
        result = predict_one(model, values)
    except ValueError as error:
        st.error(str(error))
    except Exception as error:
        st.error(f"Prediction could not be completed: {error}")
    else:
        st.subheader(result["label"])
        st.write(f"Model class: {result['predicted_class']}")
        st.caption(result["notice"])
        output = pd.DataFrame([result])
        st.dataframe(output)
        st.download_button(
            "Download this result as CSV",
            output.to_csv(index=False).encode("utf-8"),
            file_name="step5_prediction.csv", mime="text/csv",
        )

with st.expander("How this demonstration works"):
    st.write("The interface validates inputs and calls predict() on the saved pipeline.")
    st.write("The pipeline already includes its fitted scaler. No training runs here.")
```

Appendix C: app.py (continued)

```
    st.write("SC11 is a reference project. Each team must use its own inputs and method.")
```

Appendix D: test_product_predict.py (project root)

SC11 worked example for learning: inspect its structure and purpose, then design or adapt a solution for your own project. The water fields, model, filenames and results are not requirements for other projects.

```
import unittest
from src.product_predict import FEATURES, validate_inputs


class ValidationTests(unittest.TestCase):
    def setUp(self):
        self.values = {name: "1" for name in FEATURES}
        self.values["ph"] = "7"

    def test_normal(self):
        self.assertEqual(validate_inputs(self.values)["ph"], 7.0)

    def test_ph_boundaries(self):
        for value in ("0", "14"):
            self.values["ph"] = value
            self.assertEqual(validate_inputs(self.values)["ph"], float(value))

    def test_invalid_ph(self):
        for value in ("-1", "15", "", "abc", "nan", "inf"):
            self.values["ph"] = value
            with self.assertRaises(ValueError):
                validate_inputs(self.values)

    def test_negative_measurement(self):
        self.values["Turbidity"] = "-1"
        with self.assertRaises(ValueError):
            validate_inputs(self.values)

    def test_missing_feature(self):
        self.values.pop("Hardness")
        with self.assertRaises(ValueError):
            validate_inputs(self.values)


if __name__ == "__main__":
```

Appendix D: test_product_predict.py (project root) (continued)

```
    unittest.main()
```

Appendix E: docs/STEP-5.md template

SC11 worked example for learning: inspect its structure and purpose, then design or adapt a solution for your own project. The water fields, model, filenames and results are not requirements for other projects.

```
# Step 5 - Product V1

## Team and project
- Team name: [your registered team]
- Project code and title: [your allotted project]
- Branch and commit: [actual branch and commit]

## User task
Explain who will use the interface, which inputs they provide and which decision
or output it supports. SC11 demonstrates a water-quality prediction interface;
other projects must describe their own problem and users.

## Interface and method
- Interface type: [actual framework or application]
- Entry file: [repository-relative path]
- Saved model/controller/optimiser: [actual path and version]
- Input schema and units: [list your project fields]
- Output labels, units and meaning: [explain]
- Data flow: input -> validation -> saved method -> explained output.

## Run instructions
Record the tested Python version, dependency installation command, working
directory, start command and expected local address/window. State whether a
local demonstration or hosted demo is available. Do not publish secrets.

## Actual test evidence
Replace these prompts with measured observations; do not invent passing results.

| Case | Actual input or evidence path | Expected behaviour | Observed behaviour | Pass/Fail |
|---|---|---|---|---|
| Normal | [fill] | [fill] | [fill] | [fill] |
| Boundary | [fill] | [fill] | [fill] | [fill] |
| Invalid or missing input | [fill] | [fill] | [fill] | [fill] |
| Missing model/service | [fill] | [fill] | [fill] | [fill] |
| Same input through UI and inference code | [fill] | matching output | [fill] | [fill] |
```

Appendix E: docs/STEP-5.md template (continued)

```
## Screenshots and evidence
Link to actual screenshots under docs/evidence/step5/ and any test output.
Show the input form, a successful result and a clear validation error.

## Earlier feedback addressed
List each relevant Step 1-4 comment, the correction made and a file or commit.
State any unresolved issue honestly.

## Limitations and responsible use
Describe what the demonstration supports and what remains unverified.
For SC11, model output does not certify drinking-water safety. Use the
appropriate limits for your own project; do not reuse SC11's statement blindly.

## Team contributions
List each real member's name, work and evidence. SC11 is an instructor example
and must not claim student contributions.
```

Appendix F: src/check_product_prediction.py

SC11 worked example for learning: inspect its structure and purpose, then design or adapt a solution for your own project. The water fields, model, filenames and results are not requirements for other projects.

```
"""Compare this printed result with the same example in the interface."""
from pathlib import Path
import pandas as pd
from product_predict import ROOT, FEATURES, load_model, predict_one


def main():
    sample = pd.read_csv(ROOT / "data/processed/validation.csv", nrows=1)
    values = {name: sample.iloc[0][name] for name in FEATURES}
    result = predict_one(load_model(), values)
    output = ROOT / "data/reports/step5_ui_reference_prediction.csv"
    output.parent.mkdir(parents=True, exist_ok=True)
    pd.DataFrame([result]).to_csv(output, index=False)
    print(pd.DataFrame([result]).to_string(index=False))
    print("Saved:", output)
    print("Compare with Load one existing example in the app.")


if __name__ == "__main__":
    main()
```

Reference links

SC11 completed Step 4 branch

Student project portal

Streamlit model resource caching